UNIVERSIDAD ABIERTA INTERAMERICANA

Facultad de Tecnología Informática

Sede Rosario – Av. Pellegrini 1618



Canonical / Ubuntu
empaquetado y seguridad



Podman
caso de estudio



Open Security Labs
plataforma de laboratorios

Software libre, contenedores rootless y transferencia abierta

PLAN DE DESARROLLO TECNOLÓGICO

Segunda Entrega – Seminario de Trabajo Final

2.º cuatrimestre 2026

Línea de Investigación Aplicada y Vinculación Tecnológica
en Ingeniería de Software basada en Go, Software Libre y Podman

Alcance de esta entrega (E2-2C):
Plan de Desarrollo Tecnológico completo hasta el punto 7 de la guía: Operaciones. Se conservan los puntos 1–6, el 10 (solución tecnológica) y los anexos de la versión anterior.

El punto 7 organiza los tres frentes de investigación y transferencia sin presentar la línea como una sociedad comercial constituida.

Autor

Valentín Torassa Colombero

Legajo: B00069076-T1

Valentin.TorassaColombero@alumnos.uai.edu.ar

Cuerpo docente y dirección de investigación

Prof. María Eugenia Casco – Directora de Investigación

Prof. Santiago Enrique Roatta – Director de la línea de Seguridad: Conceptos y Aplicaciones

Rosario, octubre de 2026

Ficha de Seguimiento y Revisión

Cambios del autor

Fecha	Autor	Versión	Referencia de los cambios
2026-05-03	V. Torassa Colombero	0.1	Anexo de vinculación tecnológica y Business Model Canvas iniciales.
2026-05-18	V. Torassa Colombero	1.0 (E1)	Primera entrega: Plan de Negocios puntos 1–3 y Solución Tecnológica 10.1–10.5.
2026-06-19	V. Torassa Colombero	1.1 (E2)	Segunda entrega: consolidación de puntos 1–4, desarrollo de Segmentación e incorporación del logo de la solución.
2026-07-07	V. Torassa Colombero	1.2 (E3)	Tercera entrega: desarrollo del Plan de Acción (punto 5), extensión de la identidad visual con Open Security Labs e incorporación de las contribuciones de seguridad a Ubuntu (`vt-security-fixes`).
2026-08-02	V. Torassa Colombero	1.3 (E3 rev.)	Adecuación a la plantilla de la guía: incorporación del logo institucional UAI en carátula y encabezado, y del número de legajo del autor.
2026-08-02	V. Torassa Colombero	1.4 (E3 rev. 2)	Actualización de evidencia: envío de los tres papers a CACIC 2026, *merge* del PR #28637 en `containers/podman` (23-06-2026) y crecimiento comunitario de Open Security Labs (32 labs, contribuyentes externos).
2026-08-02	V. Torassa Colombero	1.5 (EF)	Entrega Final. Ampliación del encuadre de negocio a tres frentes (contribución *upstream*, seguridad de paquetes y formación abierta), con Podman *rootless* como vertical de profundidad. Incorporación de un diagrama de casos de uso por cada CU y de la ficha CU-001-006. Registro de la difusión pública de la línea (P5).
2026-09-22	V. Torassa Colombero	1.6 (E1-2C)	Primera entrega del segundo cuatrimestre. Incorporación del **punto 6, Plan de Marketing** completo (6.1 a 6.7.6), elaborado sobre el Anexo de Vinculación Tecnológica y el Business Model Canvas, y sostenido en el encuadre de transferencia: objetivos como activos verificables, precio por alcance con paquetes abiertos, comunicaciones integradas sobre evidencia pública y distribución selectiva con gestión *just in time* del material.

Fecha	Autor	Versión	Referencia de los cambios
2026-10-14	V. Torassa Colombero	1.7 (E2-2C)	Se incorpora el punto 7 (Operaciones, 7.1–7.6), se actualiza el alcance de la portada y se corrige el horizonte de entregas del segundo cuatrimestre.

Revisiones

Auditor	Versión aprobada	Responsabilidad/Rol	Fecha
Prof. María Eugenia Casco	1.5 (EF)	Directora de Investigación – coherencia con la línea de I+D.	2026-08-02
Prof. Santiago E. Roatta	1.5 (EF)	Director de la línea de Seguridad: Conceptos y Aplicaciones – revisión técnica.	2026-08-02
Prof. María Eugenia Casco	1.6 (E1-2C)	Aprobación de los hitos RRR, PDR y CDR.	2026-09-22
Prof. Santiago E. Roatta	1.6 (E1-2C)	Aprobación de los hitos RRR, PDR y CDR.	2026-09-22

Descripción del Documento

Ítem	Detalle
Título del documento	Plan de Desarrollo Tecnológico – Segunda Entrega (2.º cuatrimestre).
Autor	Valentín Torassa Colombero.
Fecha de Creación	3 de mayo de 2026.
Última Actualización	14 de octubre de 2026.

Nota de encuadre. Este documento mantiene el eje definido para el Trabajo Final: investigación aplicada en Ingeniería de Software. La parte de negocio se desarrolla como vinculación tecnológica, es decir, como transferencia de conocimiento mediante talleres, guías, laboratorios reproducibles, diagnósticos y pilotos. No se plantea una startup ni un producto cerrado.

Índice

Ficha de Seguimiento y Revisión 2

A. Resumen Ejecutivo 7

B. Proyecto de Base Tecnológica 8

1. Descripción General 8
- 1.1. Descripción básica del negocio . . . 8
- 1.2. Situación actual del negocio . . . 8
- 1.3. ¿Qué hace **único** al proyecto? . . . 9
- 1.4. Factores principales de éxito esperado . . . 10
- 1.5. Estrategia: misión, visión y propósito estratégico . . . 10
- 1.6. Identificación de la oportunidad de negocio . . . 10
- 1.7. Capacidades centrales . . . 10
- 1.8. Propuesta de valor para el cliente . . . 10
- 1.9. Valores nucleares de la organización . . . 10
- 1.10. Enfoque e iniciativas estratégicas . . . 11
- 1.11. Áreas clave de resultados . . . 11
- 1.12. Ingreso al sector: estrategias de inserción . . . 11

2. Análisis Estratégico 11
- 2.1. Análisis de Contexto . . . 11
  - 2.1.1. Descripción del escenario local . . . 11
  - 2.1.2. Factores económicos . . . 11
  - 2.1.3. Factores políticos . . . 11
  - 2.1.4. Factores tecnológicos . . . 12
  - 2.1.5. Descripción del escenario–meta . . . 12
  - 2.1.6. Análisis sectorial: oportunidades y amenazas . . . 12
- 2.2. Análisis de la Competencia . . . 12
  - 2.2.1. Principales competidores directos . . . 12
  - 2.2.2. Análisis de cadena de valor . . . 13
  - 2.2.3. Factores Críticos de Éxito . . . 13
  - 2.2.4. Fortalezas y debilidades del negocio . . . 13

3. Análisis FODA 13
- 3.1. Cuadro FODA . . . 13
- 3.2. Análisis de las Fortalezas, Oportunidades, Debilidades y Amenazas . . . 14
- 3.3. Conclusión: atractivo de la industria y fortalezas del negocio . . . 14

4. Segmentación 14
- 4.1. Segmentación de consumidores y/o negocios . . . 14
- 4.2. Identificación de grupos diferenciados . . . 14
- 4.3. ¿Quiénes son los potenciales usuarios/compradores del negocio? . . . 15
- 4.4. Pautas de comportamiento esperado de cada segmento . . . 15
- 4.5. Segmento objetivo inicial . . . 15
- 4.6. Matriz de atractivo y prioridad . . . 16

5. Plan de Acción **16**
5.1. Programas generales de acción . . . 16
5.2. Programas específicos de acción . . . 17
5.3. Activos de contribución y plataforma . . . 18
5.3.1. Open Security Labs (P3) . . . 18
5.3.2. Contribuciones de seguridad a Ubuntu: `vt-security-fixes` (P2) . . . 19
5.3.3. Difusión y validación pública (P5) . . . 20

6. Plan de Marketing **21**
6.1. Objetivos . . . 21
6.2. Resultados esperados en materia de cobertura y participación . . . 21
6.3. Metas de posicionamiento . . . 22
6.4. Producto. Estrategia de producto . . . 22
6.4.1. Descripción del producto/servicio: qué es y qué no es . . . 22
6.4.2. ¿Es un producto/servicio durable o estacional? . . . 23
6.4.3. Características que influyen sobre la decisión de adopción . . . 24
6.4.4. ¿Existen estudios que respalden estas hipótesis? . . . 24
6.5. Precio. Estrategia de precio . . . 24
6.5.1. Condicionantes del precio . . . 24
6.5.2. Estrategia de precio . . . 25
6.6. CIM. Comunicaciones integradas de marketing . . . 25
6.6.1. Planificación estratégica de las acciones de comunicación . . . 25
6.6.2. Diseño del sitio web: las siete “C” . . . 26
6.7. Distribución . . . 26
6.7.1. Factores condicionantes y canales principales . . . 26
6.7.2. Estrategia de distribución . . . 26
6.7.3. Canales: tipo y nivel . . . 27
6.7.4. Localización de puntos propios . . . 27
6.7.5. Opciones de asociación . . . 27
6.7.6. Gestión *just in time* . . . 27

7. Operaciones **28**
7.1. Organización de la iniciativa . . . 28
7.2. Grupo promotor, dirección y titularidad . . . 29
7.3. Funciones críticas y perfiles . . . 29
7.4. Estructura al lanzamiento y evolución . . . 30
7.5. Filosofía y sistema de trabajo . . . 31
7.6. Requerimientos de recursos humanos por etapa . . . 31

10. Aspectos Descriptivos de la Solución Tecnológica **32**
10.1. Carátula y datos descriptivos . . . 32
10.2. Índice . . . 32
10.3. Historial de Revisión . . . 32
10.4. Documento Visión . . . 32
10.4.1. Propuesta de valor del proyecto . . . 32
10.4.2. Especificaciones de requerimientos . . . 33
10.4.3. Alcance del desarrollo . . . 33
10.4.4. Definiciones, acrónimos y abreviaciones . . . 34
10.4.5. Descripción del entorno del sistema . . . 34
10.5. Descripción Global del Producto . . . 34
10.5.1. Mapa del marco metodológico de siete fases . . . 34

10.5.2. Índice de casos de uso . . . 35
10.5.3. Casos de Uso . . . 35
10.5.4. Diagrama de secuencia . . . 40
10.5.5. Diagrama de paquetes . . . 41
10.5.6. Diagrama de componentes . . . 42
10.5.7. Diagrama de Clases (modelo de dominio de Podman) . . . 42
10.5.8. Diagrama de Entidad Relación . . . 43
10.5.9. Diagrama de despliegue . . . 44
10.5.10. Especificación de Casos de Prueba (extracto) . . . 44
10.6. Trazabilidad de requerimientos . . . 45
10.6.1. Criterios de aceptación . . . 45
10.6.2. Matriz de trazabilidad requerimiento → caso de uso . . . 45
10.7. Diagrama general de casos de uso . . . 46
10.8. Cursos básicos, alternativos y excepciones . . . 46
10.9. Prototipo de interfaz . . . 48
10.10. Diagramas de robustez . . . 49
10.11. Diagramas de secuencia por caso de uso . . . 51
10.12. Diagrama de clases de diseño . . . 53
10.13. Patrones de diseño aplicados . . . 54
10.14. Testing estructural . . . 55
10.14.1. Unidad bajo prueba y dominio de entrada . . . 55
10.14.2. Grafo de flujo de control . . . 55
10.14.3. Criterio de cobertura y conjunto de pruebas . . . 55
10.14.4. Cobertura medida y trazabilidad . . . 56
10.15. Registro de hitos RRR, PDR y CDR . . . 57

C. Anexos **57**

11. Anexos **57**
11.1. Normas de Calidad . . . 57
11.2. Anexos específicos del proyecto . . . 57
11.3. Evidencia upstream: contribuciones al repositorio Podman . . . 58
11.4. Bibliografía Consultada . . . 59

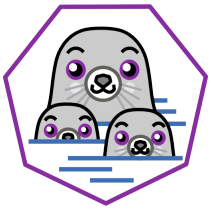
A. Resumen Ejecutivo

Concepto del proyecto

La propuesta consiste en consolidar una **línea de investigación aplicada en Ingeniería de Software** centrada en tres ejes complementarios: Go como herramienta de diseño para reducir complejidad accidental, contribuciones a proyectos de software libre como estrategia formativa y análisis funcional de Podman como motor de contenedores rootless. Sobre esa base se construye una línea de vinculación tecnológica sobre software libre y seguridad, desplegada en tres frentes: contribución *upstream* (Podman), mantenimiento y seguridad de paquetes (Ubuntu) y formación abierta reproducible (Open Security Labs).

La solución no es un sistema SaaS ni una academia masiva. Es un conjunto organizado de activos verificables: publicaciones, laboratorios reproducibles, repositorios didácticos, contribuciones a Podman y servicios de transferencia. La salida económica posible se canaliza por servicios profesionales acotados, una vez validada la utilidad del material.

Logo e identidad visual de la solución



Podman
caso de estudio principal



Canonical / Ubuntu
empaquetado y seguridad



Open Security Labs
plataforma de laboratorios

Figura 1: Identidad visual de referencia. Podman funciona como caso de estudio principal; Canonical/Ubuntu representa la continuidad del ecosistema de empaquetado y mantenimiento de seguridad; y Open Security Labs es la plataforma abierta donde se publican los materiales y laboratorios de la línea.

El uso del logo de Podman refuerza que la solución tecnológica se apoya en un ecosistema de software libre existente. El logo de Canonical/Ubuntu aparece como referencia secundaria porque la misma línea de mantenimiento open source que motiva el caso Podman se aplica también al empaquetado de distribuciones y al mantenimiento de seguridad. El logo de Open Security Labs identifica la plataforma propia – abierta y en español – donde se publican los materiales, laboratorios y evidencias de la línea. En este Trabajo Final, la identidad visual no busca crear una marca comercial independiente, sino asociar los materiales, laboratorios y contribuciones con proyectos abiertos reales.

Visión, misión y propósito estratégico

- **Visión:** ser una referencia regional acotada en contribución a software libre, seguridad de paquetes y formación técnica abierta en español, con respaldo académico. Los contenedores *rootless* con Podman son la vertical de mayor profundidad técnica.
- **Misión:** convertir investigación académica sobre Go, software libre y Podman en materiales reproducibles publicados en abierto, capacitaciones aplicadas y contribuciones reales a proyectos *upstream* activos, tanto de código (Podman) como de empaquetado y seguridad (Ubuntu).
- **Propósito estratégico:** cerrar la brecha entre el aprendizaje universitario de contenedores y la práctica de mantenimiento real de un proyecto open source maduro.

Ventajas competitivas iniciales

- Producción académica: trabajo WICC sobre Go como lenguaje de diseño (presentado y certificado) y **tres papers enviados a CACIC 2026** (software libre, análisis de Podman y parches SRU en Ubuntu).
- Evidencia verificable: contribuciones públicas a `containers/podman`, con el caso testigo PR #28637 (`-retry` en `podman manifest push`) **mergeado el 23 de junio de 2026**.
- Continuidad aplicada: la misma línea de estudio sobre software libre, mantenimiento y seguridad derivó en el repositorio `vt-security-fixes`, con parches DEP-3 y debdiffs para CVEs en paquetes `universe` de Ubuntu, asociados a bugs de Launchpad.
- Foco diferenciado: software libre con evidencia auditable en dos ecosistemas – contribución *upstream* (Podman *rootless*, Quadlet) y mantenimiento de seguridad (Ubuntu, flujo SRU) – con trazabilidad académica.
- Plataforma propia con tracción medible: Open Security Labs, seis rutas y 32 laboratorios reproducibles, con más de 1.500 visitantes únicos en 30 días y tres contribuyentes externos.
- Bajo costo operativo: el toolchain central es software libre.

B. Proyecto de Base Tecnológica en un Entorno Distribuido Habilitado para la Web

1. Descripción General

1.1 Descripción básica del negocio

La iniciativa se estructura como una **línea profesional de investigación aplicada con servicios de transferencia tecnológica**. Su salida económica se canaliza por servicios profesionales digitales B2B y B2I: talleres en línea, materiales reproducibles publicados en GitHub, diagnósticos técnicos remotos y acompañamiento. Las unidades de transferencia se agrupan en tres familias, una por frente: formación abierta y a medida sobre los materiales publicados; diagnóstico de dependencias y preparación de parches de seguridad; y pilotos de adopción de contenedores *rootless*.

- **Tipo de E-Business:** servicios profesionales digitales sobre Ingeniería de Software aplicada.
- **Modalidad de E-Commerce:** contratación por proyecto, sin tienda transaccional ni venta de licencias.
- **Activos digitales centrales:** repositorios públicos, laboratorios, documentación Markdown, publicaciones y registro de contribuciones upstream.
- **Identidad visual de la solución:** logos de Podman, Canonical/Ubuntu y Open Security Labs, uno por cada frente de la línea (Figura 1).

1.2 Situación actual del negocio

El proyecto se encuentra en etapa preliminar formativa. La línea de investigación está definida, existe producción académica inicial y se cuenta con un primer encuadre de vinculación tecnológica. La oferta de servicios todavía no se comercializó de forma sostenida; por eso el documento evita presentar ventas hipotéticas como evidencia.

Tabla 2: Estado de avance de los activos de la línea (al 2 de agosto de 2026).

Activo	Estado	Evidencia
Paper WICC sobre Go	Presentado	Certificado WICC, paper y poster en carpeta `WICC/`.
Paper CACIC contribuciones SL/Podman	Enviado (CACIC 2026)	`CACIC/Nuevas Investigaciones 2026/01-contribuciones-software-libre/`.
Paper CACIC análisis funcional Podman	Enviado (CACIC 2026)	`CACIC/Nuevas Investigaciones 2026/02-analisis-funcional-podman/`.
Paper CACIC parches SRU en Ubuntu	Enviado (CACIC 2026)	`CACIC/Nuevas Investigaciones 2026/03-parches-seguridad-comunitarios/`.
Anexo de vinculación tecnológica	Versión 1.0	`Business Idea/Anexo vinculacion tecnologica/main.pdf`.
Business Model Canvas	Versión 1.0	`canvas_modelo_negocio.pdf`.
Identidad visual	Versión inicial	Logo de Podman incorporado en esta entrega.
Contribución upstream	**Mergeada**	PR #28637 mergeado en `containers/podman` el 23-06-2026 (mantenedor `Luap99`).
Ubuntu security fixes	Repositorio complementario	Parches y debdiffs para `libsoup3`, `libyaml-syck-perl` y `gdcm`; bugs Launchpad #2157576, #2157577 y #2157578.
Open Security Labs	En producción	Seis rutas y 32 laboratorios reproducibles; más de 1.500 visitantes únicos en 30 días y tres contribuyentes externos.
Difusión pública de la línea	Realizada	Transmisión en vivo en canal propio y publicaciones en LinkedIn, X, Discord, Instagram y TikTok (Sección 5.3.3).

Como continuidad práctica de esta línea, también se relevó el repositorio personal de fixes de seguridad para Ubuntu. Ese trabajo no reemplaza el eje Podman, pero muestra que la metodología de lectura de proyectos abiertos, seguimiento de CVEs, preparación de parches y documentación para revisión comunitaria se puede trasladar a otros ecosistemas. En ese repositorio se prepararon fixes para `libsoup3` (CVE-2025-11021 y CVE-2025-4945), `libyaml-syck-perl` (CVE-2025-11683) y `gdcm` (CVE-2025-11266), con referencias a Launchpad y materiales pensados para SRU.

1.3 ¿Qué hace único al proyecto?

Las ventajas competitivas principales se sintetizan en cinco puntos:

1. **Investigación previa propia.** La línea cuenta con continuidad entre WICC, CACIC y la materia Seminario de Trabajo Final.
2. **Acoplamiento con dos ecosistemas *upstream* activos.** Podman aporta código real, revisiones reales y reglas reales de contribución; Ubuntu aporta el circuito de empaquetado y mantenimiento de seguridad (DEP-3, debdiff, SRU). Ninguna de las dos vías depende de la otra.
3. **Vertical de profundidad en *rootless*.** La propuesta se diferencia de cursos genéricos de Docker al concentrarse en seguridad operativa, *user namespaces*, Quadlet y flujos sin *daemon*. Es la línea donde el trabajo alcanza mayor detalle técnico, no el límite de la oferta.
4. **Ética documentada del uso de LLMs.** Se respeta la política del proyecto Podman: la asistencia puede existir, pero la comunicación pública y la justificación técnica deben ser propias.
5. **Plataforma abierta propia con audiencia real.** Open Security Labs publica los materiales sin registro ni costo y ya registra más de 1.500 visitantes únicos en 30 días y tres contribuyentes externos con *pull requests* mergeados. Es evidencia de demanda, no una proyección.

1.4 Factores principales de éxito esperado

- Calidad técnica verificable mediante PRs aceptados o revisados en Podman.
- Calidad académica respaldada por publicaciones y dirección de investigación.
- Reproducibilidad: cada laboratorio debe declarar versión de Podman, entorno y pasos.
- Honestidad metodológica: distinguir lo validado, lo estimado y lo pendiente.
- Costos operativos contenidos por uso de software libre.

1.5 Estrategia: misión, visión y propósito estratégico

Visión: en un horizonte 2026–2028, convertirse en una referencia acotada en Rosario-Litoral en contribución a software libre, mantenimiento de seguridad y formación abierta en español, con Podman *rootless* como vertical de mayor profundidad técnica.
Misión: transformar resultados de investigación en Ingeniería de Software en activos transferibles y publicados en abierto: guías, laboratorios, contribuciones reales a dos ecosistemas *upstream* y servicios profesionales acotados.
Propósito estratégico: institucionalizar un puente verificable entre universidad y proyectos de software libre.

1.6 Identificación de la oportunidad de negocio

La oportunidad surge de cinco fuentes:

- **Migración Docker a Podman:** organizaciones buscan alternativas abiertas y compatibles con flujos OCI.
- **Seguridad operativa:** *rootless* y principio de menor privilegio son cada vez más relevantes.
- **Presión sobre la cadena de suministro de software:** crecen los CVE en paquetes de distribución sin mantenimiento activo y la exigencia de parches trazables y documentados.
- **Déficit formativo regional:** abundan contenidos genéricos sobre contenedores, pero hay poca formación específica en Podman *rootless*, Quadlet y contribución *upstream*, y escasea el material técnico abierto y reproducible en español.
- **Demanda de evidencia profesional:** estudiantes y perfiles junior necesitan demostrar experiencia con repositorios reales, no solo ejercicios aislados.

1.7 Capacidades centrales

- Desarrollo en Go y comprensión del diseño del lenguaje.
- Análisis funcional y arquitectónico de Podman.
- Metodología de contribución a software libre.
- Experiencia previa en seguridad informática y análisis técnico.
- Redacción académica y producción de material reproducible.
- Capacidad de diseño de laboratorios prácticos.

1.8 Propuesta de valor para el cliente

> *Acompañar a estudiantes, equipos técnicos e instituciones en la adopción práctica de Go, software libre y Podman, usando investigación académica como base y materiales reproducibles como resultado.*

1.9 Valores nucleares de la organización

- Rigor académico y honestidad metodológica.
- Software libre, documentación abierta y trazabilidad.
- Reproducibilidad antes que marketing superficial.
- Comunicación responsable frente al uso de LLMs.
- Mejora continua a partir de revisiones comunitarias.

1.10 Enfoque e iniciativas estratégicas

- **Investigación-acción:** cada experiencia de laboratorio o piloto vuelve como insumo académico.
- **Microservicios de transferencia:** talleres, diagnósticos, pilotos y acompañamiento como unidades acotadas.
- **Validación temprana:** antes de escalar comercialmente, exponer los materiales a uso real y feedback abierto – publicación sin registro, difusión por canales propios y recepción de contribuciones externas.
- **Difusión técnica:** CACIC, WICC, meetups, GitHub y blog técnico.

1.11 Áreas clave de resultados

- Publicaciones académicas y presentaciones en congresos.
- Contribuciones aceptadas o revisadas en Podman.
- Parches de seguridad preparados para revisión en ecosistemas abiertos, incluyendo Ubuntu/Launchpad.
- Laboratorios reproducibles publicados.
- Estudiantes o profesionales capacitados.
- Pilotos completados con retroalimentación documentada.

1.12 Ingreso al sector: estrategias de inserción

La estrategia general es **enfoque diferenciado**. No se compite contra cursos masivos ni consultoras DevOps generalistas, sino en un nicho definido por la evidencia: software libre y seguridad con trazabilidad pública y auditable – contribución *upstream*, mantenimiento de paquetes y formación abierta en español –, con Podman *rootless* como vertical de mayor profundidad técnica. La inserción se plantea como *land and expand*: publicación abierta de materiales, difusión por canales propios y comunidad, evidencia pública verificable y luego servicios por alcance.

2. Análisis Estratégico

2.1 Análisis de Contexto

2.1.1 Descripción del escenario local

El lanzamiento se proyecta desde Rosario, Santa Fe, con cobertura inicial hacia Santa Fe, Rosario y Buenos Aires. La región combina universidades con carreras informáticas, empresas de software, equipos backend, comunidades técnicas y oportunidades de extensión universitaria.

2.1.2 Factores económicos

- Inflación y volatilidad cambiaria dificultan precios fijos en pesos.
- El sector software mantiene demanda de perfiles técnicos y capacitación especializada.
- Para servicios B2B conviene presupuestar en USD o con cláusula de actualización.

2.1.3 Factores políticos

- La Ley de Economía del Conocimiento y programas de I+D pueden funcionar como marco de apoyo.
- Las políticas públicas hacia software libre e innovación son variables, por lo que no deben asumirse como ingreso garantizado.
- La universidad funciona como espacio de validación institucional y no como garante comercial.

2.1.4 Factores tecnológicos

- Maduración de estándares OCI y herramientas cloud-native.
- Crecimiento de Go en infraestructura y herramientas de sistemas.
- Mayor interés por rootless containers, cgroups v2, user namespaces y reducción de privilegios.
- Necesidad de alternativas livianas a Kubernetes para escenarios PyME, donde Podman + Quadlet puede ser suficiente.
- Mayor exigencia sobre la cadena de suministro de software: seguimiento de CVE, SBOM y parches trazables en paquetes de distribución.

2.1.5 Descripción del escenario–meta

Escenario meta 2026–2028: Podman se consolida como herramienta conocida en pymes Linux y espacios académicos; los contenidos sobre contenedores *rootless* y sobre mantenimiento de seguridad ingresan a materias de Sistemas Operativos, Redes, Seguridad e Ingeniería de Software; Open Security Labs se consolida como referencia de material técnico abierto en español; y la línea de transferencia se posiciona como una propuesta acotada, seria y verificable.

2.1.6 Análisis sectorial: oportunidades y amenazas

Tabla 3: Lectura sectorial resumida.

Variable	Oportunidad	Amenaza
Sustitutos	Tutoriales gratuitos pueden atraer usuarios al tema.	Contenido gratuito compite contra talleres pagos.
Competidores	Pocos actores locales que combinen software libre, seguridad y formación abierta en español.	Red Hat y consultoras DevOps tienen mayor reputación comercial.
Compradores	Pymes necesitan resolver problemas prácticos.	Presupuesto limitado y ciclos de decisión lentos.
Tecnología	Toolchain abierto y reproducible.	Cambios *upstream* pueden dejar material desactualizado; se mitiga distribuyendo el riesgo entre tres frentes.
Academia	Papers y congresos dan legitimidad.	Ritmo académico puede ser más lento que la demanda del mercado.

2.2 Análisis de la Competencia

2.2.1 Principales competidores directos

- **Capacitación oficial Red Hat:** alto reconocimiento, precio elevado, foco enterprise.
- **Cursos masivos en línea:** bajo costo, foco Docker, calidad variable y poca trazabilidad académica.
- **Consultoras DevOps:** capacidad comercial alta, foco Kubernetes/cloud y no necesariamente Podman rootless.
- **Comunidades técnicas:** contenido gratuito y valioso, pero sin estructura de servicio ni acompañamiento sostenido.
- **Plataformas de formación práctica en seguridad:** alto reconocimiento y buena didáctica, pero de pago, en inglés y con laboratorios cerrados. No hay equivalente abierto y en español.

2.2.2 Análisis de cadena de valor

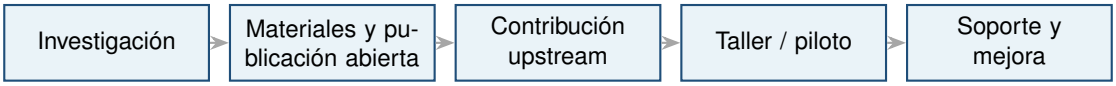


Figura 2: Cadena de valor de la línea de I+D y transferencia en software libre y seguridad.

2.2.3 Factores Críticos de Éxito

1. Credibilidad académica verificable.
2. Calidad técnica medible por revisiones, pruebas y PRs.
3. Reproducibilidad de los materiales.
4. Actualización continua respecto de los proyectos *upstream* seguidos (Podman, Ubuntu) y de los materiales publicados.
5. Capacidad de explicar valor sin lenguaje innecesariamente complejo.

2.2.4 Fortalezas y debilidades del negocio

La fortaleza central es la evidencia pública verificable en dos ecosistemas *upstream* y una plataforma abierta con audiencia propia. La debilidad central es la capacidad operativa unipersonal, que limita el volumen de servicios simultáneos. El detalle por cuadrante se desarrolla en el FODA de la Sección 3.

3. Análisis FODA

3.1 Cuadro FODA

Fortalezas
- Cuatro trabajos académicos conectados (WICC y tres CACIC).
- Evidencia en dos ecosistemas: PR mergeado en Podman y parches SRU en Ubuntu.
- Plataforma propia con audiencia: 32 labs y más de 1.500 visitantes en 30 días.
- Vinculación UAI, dirección de investigación y bajo costo (software libre).
- Política clara frente al uso de LLMs.

Oportunidades
- Adopción de software libre en PyME y sector público.
- Presión de seguridad sobre la cadena de suministro.
- Migración Docker a Podman y demanda de capacitación rootless.
- Escasez de material técnico abierto en español.
- Portfolios verificables y espacios de extensión universitaria.

Debilidades
- Etapa inicial sin pilotos pagos validados.
- Capacidad operativa unipersonal.
- Marca todavía no instalada.
- Dependencia de tiempos de revisión upstream (Podman y Ubuntu).
- Sin presupuesto relevante de marketing.

Amenazas
- Competidores con respaldo corporativo.
- Volatilidad macroeconómica.
- Contenido gratuito o generado automáticamente.
- Cambios regulatorios e impositivos.
- Concentración en plataformas gestionadas y cerradas.

Figura 3: Cuadro FODA de la línea de transferencia en software libre y seguridad.

3.2 Análisis de las Fortalezas, Oportunidades, Debilidades y Amenazas

- **FO (ofensivas):** aprovechar la producción académica para posicionarse temprano en software libre y seguridad. Acción: difundir papers, laboratorios y PRs de los tres frentes en GitHub, Open Security Labs y redes profesionales.
- **FA (defensivas):** diferenciarse de competidores corporativos con trazabilidad académica y contribuciones reales en dos ecosistemas. Acción: publicar bitácoras de aprendizaje y evidencia de revisión.
- **DO (reorientación):** suplir la falta de pilotos pagos con publicación abierta y difusión por canales propios. Acción: publicar los materiales en Open Security Labs y difundirlos por redes profesionales y comunidad técnica; el taller en cátedra UAI queda como paso posterior.
- **DA (supervivencia):** mantener alcance acotado. Acción: no aceptar pilotos largos hasta validar capacidad operativa.

3.3 Conclusión: atractivo de la industria y fortalezas del negocio

La capacitación genérica en contenedores está saturada. En cambio, la intersección entre software libre, seguridad y formación abierta en español – con Podman *rootless* como vertical de profundidad – tiene menor oferta especializada local. Los tres frentes se refuerzan entre sí: la contribución *upstream* da credibilidad, la plataforma abierta da audiencia y la línea de seguridad concentra la demanda con mayor disposición de pago. La oportunidad es atractiva si se sostiene la evidencia técnica y se evita prometer servicios que excedan la capacidad inicial.

4. Segmentación

4.1 Segmentación de consumidores y/o negocios

Se identifican cuatro segmentos primarios. La segmentación no se construye solo por capacidad de pago, sino por cercanía con el problema técnico y utilidad de la transferencia.

Tabla 4: Segmentos primarios de la línea.

Segmento	Necesidad principal	Servicio inicial sugerido	Canal de captación
Estudiantes y perfiles junior	Aprender con casos reales y construir portfolio verificable.	Taller introductorio + acompañamiento de primera contribución.	Open Security Labs, universidades, comunidades técnicas y redes profesionales.
Equipos de desarrollo PyME	Ordenar entornos de contenedores y sostener el mantenimiento de seguridad de sus dependencias.	Diagnóstico técnico (contenedores o dependencias) + piloto rootless.	Referidos, meetups, ponencias, contacto directo.
Instituciones educativas	Contar con material aplicable para SO, redes o Ingeniería de Software.	Repositorios didácticos + clases invitadas + guías de práctica.	Red docente y dirección académica.
Proyectos open source locales	Mejorar onboarding, issues, pruebas y documentación.	Revisión de guías, triage y plantillas de contribución.	Comunidad directa y GitHub.

4.2 Identificación de grupos diferenciados

Dentro del segmento PyME, que representa el mayor potencial comercial, se distinguen cuatro grupos:

- **PyME orientada a Linux:** ya usa Docker o contenedores en servidores Linux. Tiene alta probabilidad de conversión porque comprende el problema y puede valorar rootless.
- **PyME con equipos Windows/Mac:** requiere acompañamiento en `podman machine`, compatibilidad de flujos y expectativas realistas.
- **PyME que probó Podman y abandonó:** necesita diagnóstico. Las causas frecuentes pueden estar en rootless mal configurado, NFS, cgroups, puertos o documentación insuficiente.
- **PyME sin experiencia en contenedores:** no es objetivo inicial. El costo de educación básica sería alto y diluiría el foco diferencial.

Para estudiantes y perfiles junior, se distinguen:

- **Estudiantes avanzados de sistemas:** buscan evidencia para Trabajo Final, prácticas o portfolio.
- **Perfiles backend junior:** conocen Docker superficialmente y necesitan comprender contenedores desde criterios de Ingeniería de Software.
- **Perfiles DevOps iniciales:** requieren laboratorios reproducibles y guías para no depender de comandos sueltos.

4.3 ¿Quiénes son los potenciales usuarios/compradores del negocio?

Tabla 5: Perfiles de comprador y usuario.

Perfil	Rol en la decisión	Criterio de compra o adopción
Tech lead / Head of Engineering	Comprador técnico B2B	Busca reducir riesgo operativo, documentar decisiones y validar migraciones.
Sysadmin / DevOps PyME	Usuario técnico e influenciador	Evalúa si Podman rootless resuelve problemas concretos sin agregar complejidad excesiva.
Docente universitario	Comprador o prescriptor académico	Busca material reproducible, alineado con objetivos de cátedra y defendible académicamente.
Estudiante avanzado / junior	Usuario y comprador individual	Busca aprendizaje práctico, evidencia verificable y orientación para contribuir a proyectos reales.
Mantenedor open source local	Usuario no necesariamente comercial	Busca mejorar onboarding, documentación y calidad de contribuciones.

4.4 Pautas de comportamiento esperado de cada segmento

- **Estudiantes y juniors:** deciden rápido, comparan precio, valoran certificados y evidencias concretas. Prefieren talleres cortos y repositorios reutilizables.
- **PyMEs:** tienen ciclo de decisión más largo, normalmente 4 a 8 semanas. Valoran diagnóstico breve, bajo riesgo y entregables claros.
- **Instituciones educativas:** se mueven por calendario académico. Priorizan coherencia curricular, claridad didáctica y bajo costo.
- **Proyectos open source locales:** no son un segmento comercial directo. Funcionan como espacio de validación, reputación y aprendizaje comunitario.

4.5 Segmento objetivo inicial

El segmento objetivo inicial queda definido como:

> *Estudiantes avanzados, perfiles junior y docentes de materias técnicas que necesitan material reproducible para aprender Linux, seguridad, contenedores* rootless *y contribución a software libre; y, en segunda instancia, equipos PyME con base Linux que ya utilizan contenedores y necesitan ordenar su adopción de Podman o el mantenimiento de seguridad de sus dependencias.*

La priorización responde a una razón metodológica. El proyecto todavía debe validar materiales y capacidad operativa. Por eso, el primer foco es académico-formativo, donde la retroalimentación permite mejorar los laboratorios antes de ofrecer pilotos comerciales de mayor responsabilidad.

4.6 Matriz de atractivo y prioridad

Tabla 6: Matriz de atractivo por segmento. Escala: Bajo, Medio, Alto.

Segmento	Atractivo comercial	Facilidad de acceso	Valor académico	Prioridad
Estudiantes / juniors	Medio	Alto	Alto	Alta: permite validar talleres y laboratorios.
Instituciones educativas	Medio	Medio	Alto	Alta: alinea con la materia y la dirección de investigación.
PyMEs Linux	Alto	Medio	Medio	Media-alta: ideal para E3/E4, tras validar material.
PyMEs sin contenedores	Bajo	Bajo	Medio	Baja: requiere educación básica fuera del foco inicial.
Proyectos OSS locales	Bajo	Alto	Alto	Media: aporta reputación y aprendizaje, no ingresos directos.

5. Plan de Acción

El Plan de Acción operacionaliza la estrategia de las secciones anteriores. Traduce los cruces del FODA (Sección 3) y los segmentos priorizados (Sección 4, Tabla 6) en programas concretos, con hitos verificables y evidencia pública. La lógica es *investigación* → *contribución* → *materiales* → *servicios*: cada acción produce un activo citable o auditable, en coherencia con el encuadre de vinculación tecnológica y con la línea de Seguridad de la dirección del trabajo.

Tres activos verificables sostienen el plan:

Podman (upstream)
PR #28637 **mergeado** (23-06-2026): soporte `-retry` en `podman manifest push`.

Ubuntu (seguridad)
`vt-security-fixes`: parches DEP-3 para 3 paquetes de `universe`.

Open Security Labs
Plataforma abierta: 6 rutas y 32 laboratorios, con contribuciones externas.

5.1 Programas generales de acción

Se definen cinco programas estratégicos:

- **P1 – Investigación aplicada.** Consolidar la producción académica: el trabajo WICC sobre Go (presentado) y los tres papers CACIC 2026 – contribuciones a software libre, análisis funcional de Podman y parches de seguridad comunitarios en Ubuntu – **ya enviados**. *Hito:* presentación en CACIC 2026 (Q4 2026).
- **P2 – Contribución a software libre.** Sostener dos frentes verificables: (a) Podman *upstream*, con el PR #28637 **mergeado** (23-06-2026) y un *pipeline* de contribuciones de bajo riesgo; (b) seguridad en Ubuntu, mediante el repositorio `vt-security-fixes`, que ya organiza parches DEP-3 para `libsoup3`, `libyaml-syck-perl` y `gdcm`, con bugs de Launchpad presentados. *Hito:* primer aporte SRU aceptado en `-proposed`.

- **P3 – Plataforma educativa abierta.** Publicar y ampliar Open Security Labs: seis rutas y 32 laboratorios reproducibles, con exámenes, *badges*, seguimiento de progreso y primeras contribuciones externas de la comunidad. *Hito:* lanzamiento público y crecimiento sostenido de rutas y labs.
- **P4 – Servicios de transferencia.** Operar tres familias de servicio, una por frente: (a) *formación* – talleres y rutas a medida sobre el material publicado en Open Security Labs; (b) *seguridad y mantenimiento* – diagnóstico de dependencias, seguimiento de CVE y preparación de parches, sobre la línea `vt-security-fixes`; (c) *contenedores* – pilotos *rootless* con Podman y Quadlet, y acompañamiento. Todo sobre los segmentos priorizados en el punto 4, con alcance acotado. *Hito:* catálogo con una línea por familia y primer servicio pago acotado.
- **P5 – Validación y difusión.** La primera instancia de validación ya se ejecutó por canales propios: presentación pública de la línea en una transmisión en vivo y difusión posterior en redes, con respuesta medible en Open Security Labs – más de 1.500 visitantes únicos en 30 días y tres contribuyentes externos con *pull requests* mergeados (Sección 5.3.3). Continúa con participación en *meetups* regionales (Rosario, Buenos Aires) y, más adelante, un taller en cátedra UAI. *Hito:* difusión propia con respuesta medible (cumplido); *meetup* regional y taller en cátedra como hitos siguientes.

Estos programas responden a los cruces del FODA: P1 y P3 materializan las iniciativas ofensivas (aprovechar la producción académica ante la adopción de software libre, la presión de seguridad sobre las dependencias y la migración Docker $\rightarrow$ Podman); P2 refuerza las defensivas (profundidad técnica frente a competidores corporativos); P5 ejecuta las de reorientación (suplir la falta de pilotos validados); y el alcance acotado de P4 – sin contratos mayores a cuatro semanas en el primer año – implementa la prudencia de supervivencia. La priorización por segmento sigue la matriz de atractivo (Tabla 6): foco inicial en estudiantes/juniors e instituciones educativas, y avance hacia PyMEs Linux una vez validado el material.

5.2 Programas específicos de acción

La Tabla 7 desagrega los programas en acciones con horizonte temporal e hito verificable. El horizonte se expresa por trimestre calendario y por las entregas del segundo cuatrimestre: E1 presentada, E2 en preparación y E3 prevista para noviembre.

Tabla 7: Programas específicos de acción con hitos verificables.

Prog.	Acción específica	Horizonte	Hito verificable
P1	Finalizar y enviar los borradores CACIC (contribuciones, análisis Podman, parches SRU)	2026 Q3/Q4	Constancia de envío y, luego, de exposición.
P2	PR #28637 mergeado (23-06-2026); avanzar el *pipeline* de contribuciones	2026 Q3/Q4	PR mergeado en `containers/podman` (cumplido).
P2	Presentar el primer aporte SRU de `vt-security-fixes` vía Launchpad y *sponsorship*	2026 Q4 / 2027 Q1	Bug con debdiff aceptado en `-proposed`.
P3	Lanzamiento público de Open Security Labs y ampliación de rutas/labs	2026 Q3 en adelante	Sitio en línea; incremento medible de labs.
P4	Definir catálogo de servicios y *pricing* en USD; primer diagnóstico piloto	2027 Q1/Q2	Catálogo publicado; primer servicio contratado.
P5	Presentar la línea en abierto y difundirla por canal propio y redes	2026 Q3 (cumplido)	Transmisión publicada; +1.500 visitantes únicos/30 días y 3 contribuyentes externos en Open Security Labs.
P5	Taller en cátedra UAI y *meetups* regionales	2026 Q4 / 2027 Q1	Taller dictado; encuesta procesada; registro de charlas.

La Figura 4 representa el cronograma tentativo de los programas para 2026 Q3–2027 Q4.

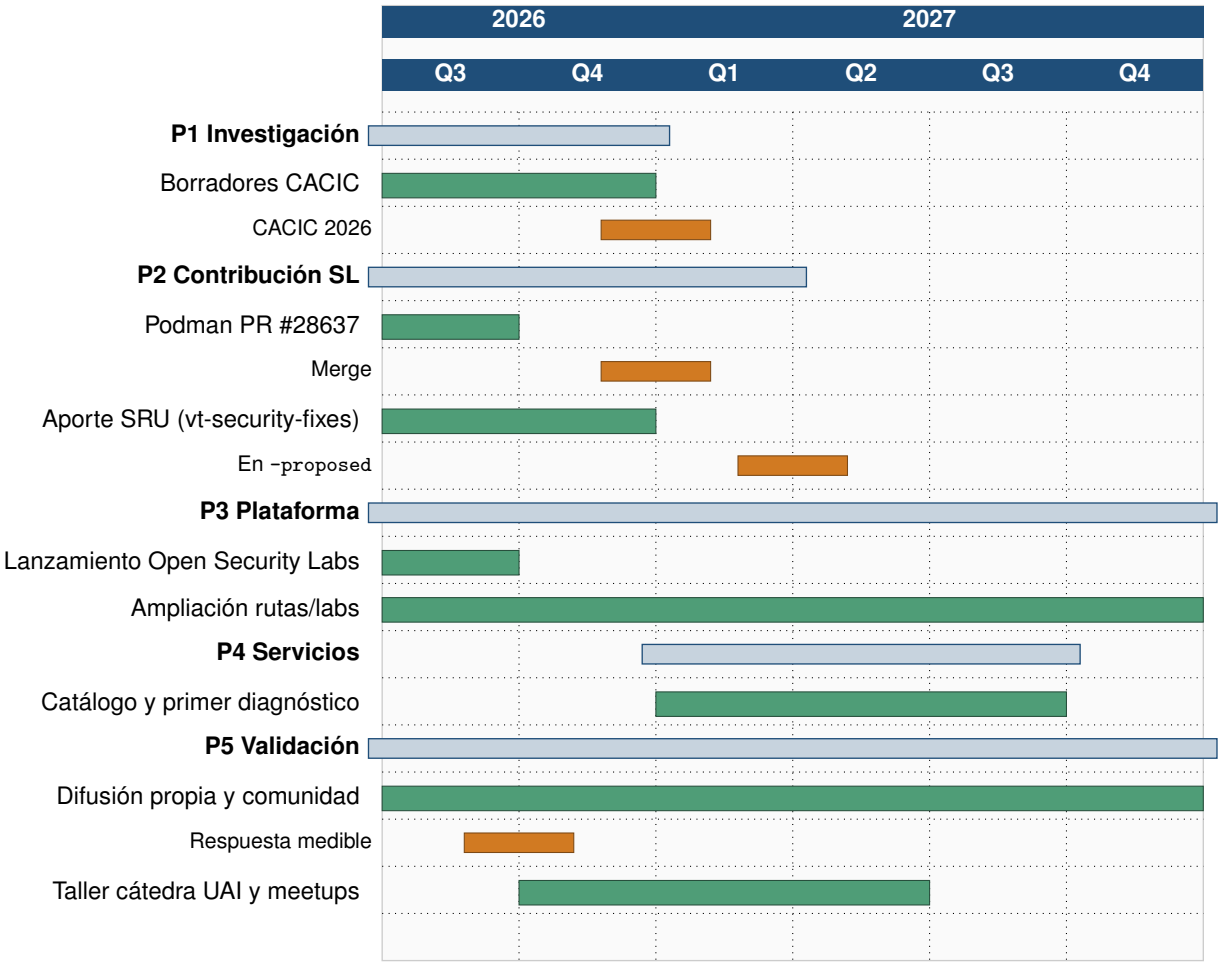


Figura 4: Cronograma tentativo de los programas de acción (2026 Q3–2027 Q4). Los rombos indican hitos verificables.

5.3 Activos de contribución y plataforma

5.3.1 Open Security Labs (P3)

Open Security Labs es un sitio estático, abierto y en español, orientado a aprender Linux, redes, backend, cloud y ciberseguridad entendiendo los sistemas de verdad. La Figura 5 muestra la portada del sitio en producción.



Figura 5: Portada de Open Security Labs (`https://securitylabs.valentorassa.com`). El contenido se organiza en seis rutas: `linux-real`, `redes-internet`, `backend-arquitectura`, `cloud-produccion`, `ciberseguridad` y `devsecops-agentes`.

Ítem	Detalle
Sitio público	`https://securitylabs.valentorassa.com`
Repositorio	`https://github.com/ValentinTorassa/Open-Security-Labs`
Tecnología	Sitio estático Astro; contenido Markdown/MDX versionado; sin backend ni login.
Rutas y labs	Seis rutas de aprendizaje; 32 laboratorios reproducibles al cierre de esta revisión.
Comunidad	Tres contribuyentes externos con PRs mergeados (contenido, correcciones y CI; serie de PRs #22–#28).
Tráfico	Más de 1.500 visitantes únicos en los últimos 30 días (medición al 2-08-2026).
Modelo	Abierto (100 % open source), progreso local, sin registro de usuarios.

Podman *rootless* se integra como contenido especializado dentro de las rutas de Linux, Cloud y DevSecOps, junto al resto de los contenidos. La plataforma cumple dos funciones en el plan: es el vehículo concreto de los materiales reproducibles que sostiene la segmentación del punto 4, y es el activo con tracción verificable de la línea – más de 1.500 visitantes únicos en 30 días y tres contribuyentes externos –, o sea el canal de captación natural de los servicios del programa P4.

5.3.2 Contribuciones de seguridad a Ubuntu: `vt-security-fixes` (P2)

El repositorio `vt-security-fixes` organiza parches de seguridad DEP-3 y artefactos SRU para paquetes de Ubuntu `universe` que no recibieron actualización en el bolsillo estándar. La Tabla 8 resume los casos incluidos.

Tabla 8: Casos incluidos en `vt-security-fixes` (`https://github.com/ValentinTorassa/vt-security-fixes`).

Paquete	CVE	Releases	Bug Launchpad
`libsoup3`	CVE-2025-11021, CVE-2025-4945	Jammy	#2157576
`libyaml-syck-perl`	CVE-2025-11683	Jammy, Noble	#2157577
`gdcm`	CVE-2025-11266	Jammy, Noble, Questing	#2157578

El repositorio provee, por paquete, parches DEP-3, reportes `bug-report.md` listos para Launchpad, debdiffs por release y scripts de verificación. Junto con el PR #28637 en `containers/podman`, constituye la evidencia de contribución a software libre del programa P2.

5.3.3 Difusión y validación pública (P5)

La primera instancia de validación de la línea no fue un piloto de cátedra sino una presentación pública por canales propios. La línea de trabajo y los laboratorios se presentaron en una transmisión en vivo en el canal de YouTube del autor, difundida luego por LinkedIn, X, Discord, Instagram y TikTok. La Tabla 9 resume el alcance registrado.

Tabla 9: Difusión pública de la línea y respuesta medible.

Ítem	Detalle
Transmisión en vivo	Presentación de la línea de investigación y de los laboratorios, en canal propio de YouTube.
Enlace público	`https://www.youtube.com/watch?v=TsCc5u-MOOA`
Audiencia en vivo	Más de 1.000 espectadores durante la emisión, con un pico de 160 espectadores simultáneos.
Reproducciones	4.500 al 2 de agosto de 2026.
Difusión complementaria	LinkedIn, X, Discord, Instagram y TikTok.
Respuesta medible	Más de 1.500 visitantes únicos en 30 días y tres contribuyentes externos con *pull requests* mergeados en Open Security Labs.

Esta evidencia cumple el criterio de honestidad metodológica del documento: no se trata de una proyección de demanda sino de un registro público y verificable por terceros, en la misma línea que el PR #28637 y los bugs de Launchpad. El taller en cátedra UAI se mantiene en el plan como instancia posterior de validación en contexto académico formal, no como primera prueba de interés.

6. Plan de Marketing

El Plan de Marketing traduce la propuesta de valor y los segmentos priorizados (Sección 5 y Sección 4) en decisiones concretas de producto, precio, comunicación y distribución. Se mantiene el encuadre del trabajo: no se trata de comercializar un producto de software, sino de ordenar la **transferencia** de una línea de investigación aplicada. Por eso, donde la guía habla de "venta" y "mercado", este plan habla de *adopción* y *ámbito de aplicación*: el resultado buscado es que los materiales sean usados, revisados y citados, y sólo en segunda instancia que sostengan servicios profesionales.

6.1 Objetivos

Los objetivos se enuncian sobre activos verificables, no sobre proyecciones de venta. Cada uno es comprobable con evidencia pública.

#	Objetivo	Evidencia de cumplimiento
O1	Consolidar la línea como investigación citable, con los tres trabajos publicados o aceptados.	Constancias de WICC y CACIC; DOI o actas.
O2	Sostener al menos una contribución aceptada en un proyecto de software libre de referencia.	Pull request integrado en el repositorio upstream.
O3	Disponer de un cuerpo de laboratorios reproducibles ejecutables por terceros sin asistencia del autor.	Repositorio público con instrucciones y ejecución verificada.
O4	Validar los materiales con cohortes reales en al menos dos contextos distintos (cátedra y equipo técnico).	Registro de la actividad y devolución escrita de los participantes.
O5	Producir un diagnóstico reutilizable de uso de contenedores que otros equipos puedan autoaplicar.	Checklist publicado y al menos una aplicación documentada.

Tabla 10: Objetivos del plan de marketing, expresados como activos verificables.

Los objetivos responden a una secuencia deliberada: primero existe la evidencia académica (O1) y la evidencia comunitaria (O2); recién sobre esa base se ofrece material (O3) y se lo valida con personas (O4–O5). Invertir ese orden supondría ofrecer transferencia sin haber demostrado la competencia que la respalda.

6.2 Resultados esperados en materia de cobertura y participación

La "cobertura" se interpreta como alcance de aplicación de los materiales, y la "participación" como el peso relativo dentro del nicho específico en el que el trabajo puede competir: la formación práctica en contenedores *rootless* y contribución a software libre en el ámbito académico argentino, un espacio hoy escasamente cubierto en castellano.

Dimensión	Etapa inicial (validación)	Etapa de consolidación
Cobertura académica	Una cátedra de la propia facultad.	Varias materias de sistemas operativos, infraestructura e ingeniería de software.
Cobertura técnica	Un equipo pequeño en piloto acotado.	Equipos de pymes tecnológicas de la región.
Cobertura comunitaria	Una contribución documentada.	Participación sostenida en el ciclo de revisión del proyecto.
Participación en el nicho	Presencia incipiente, sostenida por la publicación.	Referencia consultable en castellano para adopción *rootless*.

Tabla 11: Cobertura y participación esperadas por etapa.

No se fijan metas numéricas de facturación ni de cantidad de clientes. En esta etapa una cifra de ingresos sería una hipótesis sin respaldo, y la guía exige consistencia entre lo que se afirma y lo que se puede sostener con evidencia. Las magnitudes se incorporarán cuando exista al menos un ciclo de validación completo.

6.3 Metas de posicionamiento

La posición buscada se puede enunciar en una frase:

> *La línea de trabajo que conecta investigación académica con contribución real a software libre, y que entrega materiales reproducibles en lugar de recomendaciones genéricas.*

Alternativa	Qué ofrece	Diferencia de esta propuesta
Cursos y *bootcamps* comerciales	Contenido amplio y masivo, orientado a empleabilidad.	Acá el material es acotado, reproducible y con respaldo académico verificable.
Consultoría técnica	Resolución puntual de un problema del cliente.	Acá queda conocimiento documentado en poder del equipo, no dependencia del consultor.
Documentación oficial del proyecto	Referencia completa y actualizada.	Acá hay criterio de adopción y recorrido guiado, que la documentación de referencia no provee.
Divulgación en blogs y video	Acceso inmediato y gratuito.	Acá hay trazabilidad: cada afirmación remite a una fuente o a evidencia ejecutable.

Tabla 12: Posicionamiento frente a alternativas disponibles.

El atributo que sostiene la posición es la **verificabilidad**. Es también el más difícil de imitar para las alternativas comerciales, porque exige producir investigación y participar del ciclo de revisión de un proyecto abierto, dos actividades que no escalan por inversión publicitaria.

6.4 Producto. Estrategia de producto

6.4.1 Descripción del producto/servicio: qué es y qué no es

El producto es un **paquete de transferencia** derivado de la línea de investigación. Está compuesto por cuatro elementos que se ofrecen por separado o combinados.

Componente	Contenido	Formato de entrega
Guías de adopción	Criterios para contenedores *rootless*, Quadlet e integración con `systemd`.	Documento público versionado.
Laboratorios	Prácticas Go + Podman ejecutables y repetibles.	Repositorio con instrucciones.
Talleres	Instancia guiada sobre los laboratorios y el marco de contribución.	Presencial o remoto, por cohorte.
Diagnóstico	Relevamiento del uso de contenedores, documentación y seguridad operativa.	Informe con hallazgos y prioridades.

Tabla 13: Componentes del paquete de transferencia.

Delimitar lo que el producto **no es** resulta tan importante como describirlo, porque evita expectativas que el proyecto no puede sostener:

- No es una plataforma de software ni un servicio con suscripción.
- No es un *bootcamp* ni una certificación con validez formal.
- No es consultoría de infraestructura llave en mano ni soporte de producción.
- No es una herramienta que reemplace la documentación oficial del proyecto upstream.
- No es un emprendimiento constituido: es una línea de investigación con salida de transferencia.

6.4.2 ¿Es un producto/servicio durable o estacional?

Conviene separar el núcleo del material de su superficie técnica.

- **Núcleo durable.** El marco metodológico de contribución y los criterios de diseño no dependen de una versión puntual de la herramienta: conservan validez aunque cambie el ecosistema.
- **Superficie perecedera.** Los laboratorios dependen de versiones concretas de Podman y del sistema operativo, y se degradan si no se revisan. Requieren mantenimiento periódico y fijación explícita de versiones.
- **Estacionalidad de la demanda.** La demanda académica sigue el calendario lectivo, con picos al inicio de cada cuatrimestre y en las fechas de congresos. La demanda de equipos técnicos no es estacional, pero sí esporádica: aparece asociada a una migración o a un incidente operativo.

Esta asimetría tiene una consecuencia de planificación: el mantenimiento de los laboratorios debe presupuestarse como costo recurrente, no como esfuerzo único de producción.

6.4.3 Características que influyen sobre la decisión de adopción

Característica	Por qué influye	Peso estimado
Reproducibilidad	Permite verificar el resultado sin depender del autor.	Alto
Evidencia pública	La contribución integrada demuestra competencia real, no declarada.	Alto
Respaldo académico	La publicación en congresos habilita el uso institucional.	Alto
Licenciamiento abierto	Elimina barreras legales para el uso en cátedras.	Medio
Idioma castellano	Reduce la fricción frente a la documentación original.	Medio
Independencia de proveedor	No ata la adopción a una plataforma comercial.	Medio
Costo de entrada	Permite empezar con un alcance chico antes de comprometerse.	Medio

Tabla 14: Atributos que inciden en la decisión de adopción.

6.4.4 ¿Existen estudios que respalden estas hipótesis?

El respaldo proviene de tres fuentes, de distinta naturaleza y peso probatorio:

1. **Producción propia evaluada por pares.** Los tres trabajos de la línea (Go como lenguaje de diseño, contribuciones a software libre como estrategia de formación, y análisis funcional de Podman) fueron sometidos a evaluación externa. Constituyen la base argumentativa del punto.
2. **Evidencia de campo.** La contribución al repositorio upstream, documentada en la Sección de Anexos, funciona como prueba de que el marco propuesto es aplicable y no sólo enunciable.
3. **Documentación técnica de referencia.** La documentación oficial del proyecto y las actas de congresos previos sostienen las afirmaciones sobre el comportamiento del ecosistema.

Corresponde señalar el límite: no se dispone todavía de un estudio de demanda con relevamiento sistemático de usuarios. Las hipótesis sobre disposición a adoptar y a pagar son, por ahora, conjeturas fundadas en la observación del contexto, y así deben leerse. Validarlas es precisamente el objeto de los pilotos previstos en el Plan de Acción.

6.5 Precio. Estrategia de precio

6.5.1 Condicionantes del precio

- **El conocimiento se publica abierto.** Los materiales se licencian de forma abierta, de modo que no puede cobrarse por el acceso. Lo que puede tener precio es el *acompañamiento*: la instancia guiada, el diagnóstico y la adaptación al contexto de cada equipo.
- **Presupuesto del segmento académico.** Las instituciones educativas operan con partidas acotadas y ciclos de aprobación largos, incompatibles con precios altos o con contrataciones urgentes.
- **Disponibilidad del autor.** La capacidad de entrega está limitada por horas reales de una sola persona, lo que impide competir por volumen y obliga a un posicionamiento de alcance acotado.
- **Comparables del mercado.** Existen cursos y consultorías con precios de referencia públicos; el valor percibido de esta propuesta no está en el precio sino en la trazabilidad, de modo que competir por precio sería contraproducente.
- **Etapa de validación.** Mientras el objetivo sea obtener evidencia de utilidad, el precio cumple una función secundaria frente a la devolución documentada del participante.

6.5.2 Estrategia de precio

Se adopta una estrategia de **precio por alcance**, con paquetes discretos en lugar de tarifa horaria. El criterio evita discutir el valor del tiempo y lo traslada al entregable, que es verificable.

Paquete	Alcance	Criterio de precio
Taller introductorio	Una cohorte, laboratorios base, sin adaptación.	Precio de entrada; gratuito si la institución aporta validación documentada.
Diagnóstico técnico	Relevamiento e informe de hallazgos priorizados.	Precio fijo por alcance cerrado.
Piloto corto	Adopción acotada de contenedores *rootless* sobre un caso real.	Precio fijo, con criterio de éxito acordado por escrito.
Acompañamiento	Revisión periódica y soporte a las primeras contribuciones.	Abono mensual con tope de horas.

Tabla 15: Estructura de paquetes y criterio de fijación.

Dos decisiones explícitas acompañan la estrategia. La primera es el **precio cero deliberado** en las instancias iniciales con instituciones: no es una promoción comercial sino una inversión en evidencia, y su contraprestación es la devolución escrita que alimenta los objetivos O4 y O5. La segunda es que no se fijan importes en este documento: hacerlo antes del primer piloto produciría cifras sin sustento, contrarias al criterio de consistencia que la propia guía exige.

6.6 CIM. Comunicaciones integradas de marketing

6.6.1 Planificación estratégica de las acciones de comunicación

La comunicación se organiza sobre un único mensaje rector — *investigación aplicada con evidencia verificable* — declinado según el canal, de modo que cada público reciba la prueba que le resulta significativa.

Canal	Público	Mensaje y prueba que lo sostiene
Congresos y publicaciones	Comunidad académica, dirección del trabajo.	Rigor metodológico; la prueba es la evaluación por pares.
Repositorio y ciclo de revisión	Comunidad de software libre.	Competencia técnica real; la prueba es el cambio integrado.
Cátedras y talleres	Estudiantes y docentes.	Utilidad didáctica; la prueba es el laboratorio que se ejecuta en clase.
Sitio del proyecto	Equipos técnicos y pymes.	Aplicabilidad; la prueba es el diagnóstico autoaplicable.
Paquete para docentes	Dirección y evaluadores.	Continuidad de la línea; la prueba es el conjunto ordenado de artefactos.

Tabla 16: Matriz de comunicaciones integradas.

La integración se verifica en que ningún canal afirma algo que no pueda mostrarse en otro: lo que se enuncia en un artículo se puede ejecutar en el repositorio, y lo que se enseña en un taller remite al artículo. Se excluye deliberadamente la publicidad paga, porque no aporta a un posicionamiento cuyo activo es la verificabilidad.

6.6.2 Diseño del sitio web: las siete “C”

El sitio del proyecto se evalúa con el modelo de las siete “C” de diseño de interfaces comerciales, adaptado a un sitio de transferencia técnica.

Dimensión	Decisión de diseño adoptada
Contexto	Estética sobria y técnica, jerarquía tipográfica clara y navegación plana; prioridad a la lectura por sobre el impacto visual.
Contenido	Guías, laboratorios y publicaciones, cada pieza fechada y versionada, con el estado de vigencia visible.
Comunidad	Enlace directo al repositorio y al canal de *issues*: la discusión ocurre donde queda registrada, no en un formulario privado.
Personalización	Recorridos diferenciados según perfil (estudiante, docente, equipo técnico) sobre el mismo cuerpo de material.
Comunicación	Vía de contacto institucional y devolución pública sobre los materiales; sin captura de datos innecesaria.
Conexión	Vínculos salientes a la documentación oficial, a las actas de los congresos y a la evidencia de contribución.
Comercio	Presencia mínima y explícita: los paquetes de la Tabla 15 se describen como alcance, sin carrito ni pago en línea.

Tabla 17: Aplicación del modelo de las siete “C” al sitio del proyecto.

La Figura 5, presentada en la Sección 5, corresponde a la portada del sitio en producción y permite verificar las decisiones de *Contexto* y *Contenido* de la Tabla 17: entrada directa al material, sin intermediación comercial.

6.7 Distribución

6.7.1 Factores condicionantes y canales principales

La distribución no presenta logística física: el material es digital y su costo marginal de reproducción es prácticamente nulo. Los condicionantes son de otra naturaleza:

- **Dependencia de plataformas de terceros.** El repositorio y el sitio se apoyan en servicios que el proyecto no controla, lo que constituye un riesgo de continuidad ya registrado en el análisis FODA.
- **Calendario institucional rígido.** El canal académico sólo se abre en ventanas determinadas por el ciclo lectivo y por las fechas de los congresos.
- **Ritmo del proyecto upstream.** La incorporación de una contribución depende de tiempos de revisión comunitarios ajenos a la planificación propia.
- **Capacidad de entrega personal.** Los talleres y diagnósticos no son replicables sin la presencia del autor, lo que limita estructuralmente el alcance de esos componentes.

6.7.2 Estrategia de distribución

Se adopta una estrategia **selectiva y abierta**. Abierta para los materiales, que se publican sin restricción de acceso para maximizar cobertura y revisión externa. Selectiva para las instancias guiadas, que se concentran en pocos casos de alto valor probatorio en lugar de dispersarse en muchos contactos superficiales.

6.7.3 Canales: tipo y nivel

Nivel	Canal	Función
Nivel 0	Repositorio y sitio propio.	Distribución directa, sin intermediarios; es el canal principal del material.
Nivel 1	Institución educativa o empresa.	Intermediario que convoca a la cohorte y aporta el contexto de validación.
Nivel 2	Comunidad de software libre y congresos.	Amplificación y legitimación; no distribuye el material pero habilita su circulación.

Tabla 18: Niveles de canal y función de cada uno.

6.7.4 Localización de puntos propios

No existen puntos de venta físicos ni está previsto abrirlos. Los “puntos propios” son tres y de naturaleza distinta: el **repositorio público**, disponible de forma permanente y sin restricción horaria; el **sitio del proyecto**, que ordena el material y funciona como puerta de entrada; y el **aula** de la sede de la universidad en Rosario, único punto físico, utilizado para las instancias presenciales y disponible sólo en el marco institucional.

6.7.5 Opciones de asociación

Socio potencial	Aporte esperado	Estado
Cátedras de la facultad	Acceso a cohortes reales y encuadre institucional.	Vínculo existente.
Comunidad del proyecto upstream	Revisión técnica y legitimación de la contribución.	Vínculo iniciado.
Ecosistema de distribución Linux	Contexto de empaquetado y seguridad para los laboratorios.	Vínculo existente.
Comunidades técnicas y capitulos locales	Difusión y convocatoria a instancias abiertas.	A explorar.
Pymes tecnológicas de la región	Caso real para piloto y devolución de campo.	A explorar.

Tabla 19: Opciones de asociación y estado del vínculo.

6.7.6 Gestión *just in time*

El principio *just in time* se traslada a la producción de material: no se elabora contenido antes de que exista una instancia concreta que lo requiera. La razón es específica de este dominio: el ecosistema de contenedores cambia con rapidez, de modo que el material producido por anticipado se convierte en *inventario obsoleto* antes de ser utilizado, con el costo agregado de que un laboratorio desactualizado daña la credibilidad más de lo que aporta su disponibilidad.

De ahí tres reglas operativas: producir el laboratorio cuando la cohorte está confirmada; fijar explícitamente las versiones de cada práctica y registrar la fecha de verificación; y revisar el material vigente en cada ciclo, retirando del sitio lo que no pueda garantizarse. Es preferible ofrecer menos material verificado que un catálogo amplio sin mantenimiento.

7. Operaciones

La estrategia de los puntos anteriores requiere una organización capaz de sostener investigación, contribuciones revisables y transferencia sin confundir esas actividades con la estructura de una empresa constituida. En esta etapa la iniciativa es una línea de trabajo dirigida académicamente y coordinada por el autor. Open Security Labs es su plataforma abierta de publicación y práctica; no es una persona jurídica. Los servicios de P4 son una posibilidad acotada que se habilitaría después de validar alcance, capacidad y forma de contratación. Las estimaciones de dedicación de este capítulo son *supuestos de planificación*, no contrataciones ni disponibilidades confirmadas de terceros.

7.1 Organización de la iniciativa

La línea se articula con la investigación aplicada en Ingeniería de Software de la Facultad de Tecnología Informática de la UAI. La dirección del trabajo revisa la consistencia académica y técnica; el autor mantiene el plan operativo, los repositorios, los materiales y la trazabilidad de la evidencia. No se atribuyen a la Universidad la operación cotidiana de los repositorios ni obligaciones comerciales que no se han acordado.

La actividad se ordena en tres frentes que ya aparecen en el Plan de Acción (Sección 5):

1. **Contribución *upstream*.** Investigar, reproducir problemas, preparar cambios pequeños y someterlos a la revisión del proyecto receptor, con Podman como caso de profundidad (P1 y P2).
2. **Seguridad de paquetes.** Analizar vulnerabilidades, preparar parches y evidencia de prueba para la revisión de los mantenedores de la distribución; la aceptación externa no se presume (P2).
3. **Formación y transferencia abierta.** Mantener laboratorios reproducibles en Open Security Labs, publicar guías y, cuando haya demanda validada, ofrecer talleres, diagnósticos o pilotos de alcance cerrado (P3 y P4). La difusión y la devolución de usuarios alimentan P5.

Cada frente produce un activo verificable: una contribución revisada, un parche con pruebas, un laboratorio versionado o un informe de piloto. El autor prioriza el *backlog* común según valor formativo, riesgo técnico y capacidad disponible. Una publicación o un servicio nunca reemplaza la revisión que corresponda a un proyecto *upstream* o a la dirección del Trabajo Final.

Forma de contratación. No se ha constituido una sociedad, ni se ha acordado un vehículo institucional de facturación. Antes de aceptar el primer servicio pago de P4 deberán definirse con la contraparte las responsabilidades, el encuadre legal y fiscal aplicable, la titularidad de los entregables, la confidencialidad y el mecanismo de cobro. Por eso el cronograma (Tabla 7) ubica el primer servicio contratado recién en 2027 Q1/Q2. Ningún convenio con la UAI ni ninguna forma jurídica se da por existente en este documento.

7.2 Grupo promotor, dirección y titularidad

Persona	Vínculo comprobable	Papel en esta etapa
Valentín Torassa Colombero	Autor del Trabajo Final	Coordina el plan, realiza la investigación aplicada, desarrolla los activos abiertos y documenta la evidencia. Su experiencia reflejada en el proyecto comprende Go, Podman, parches de seguridad y diseño de laboratorios.
María Eugenia Casco	Dirección de Investigación	Revisión del encuadre de I+D y de la coherencia metodológica del Trabajo Final; no se la presenta como administradora de un negocio.
Santiago Enrique Roatta	Dirección de la línea de Seguridad: Conceptos y Aplicaciones	Revisión técnica del enfoque de seguridad; no se le asignan funciones comerciales ni de gestión diaria.

Tabla 20: Personas identificadas y alcance de su participación.

No hay directorio, accionistas ni personal contratado de esta iniciativa: esas categorías presuponen una organización societaria que hoy no existe. La dirección académica y la autoría constan en la carátula y en la ficha de seguimiento; ninguna de ellas implica participación accionaria. Si en una etapa posterior se creara una entidad de transferencia, su gobierno y titularidad requerirían una decisión y documentación separadas.

7.3 Funciones críticas y perfiles

La capacidad inicial depende principalmente del autor. La Tabla 21 separa *funciones necesarias* de *puestos cubiertos*: una misma persona puede cumplir más de una función, y los colaboradores externos sólo participarían mediante acuerdo o contribución voluntaria aceptada.

Función	Responsabilidad y resultado	Perfil mínimo	Programa
Investigación	Elegir problemas, formular preguntas, conservar fuentes y validar entregables académicos. Cubierta por el autor.	Ingeniería de Software, lectura crítica y escritura técnica.	P1, P5
Contribución	Reproducir fallas, proponer cambios, ejecutar pruebas y responder revisiones. Cubierta inicialmente por el autor.	Go, Git, pruebas y normas del proyecto receptor.	P2
Seguridad de paquetes	Trazar CVE a versión y parche, preparar debdiff/DEP-3 y evidencia; requiere revisión externa para aceptación.	Linux, empaquetado, análisis de vulnerabilidades y flujo SRU.	P2
Laboratorios	Versionar entornos, consignas y comprobaciones; retirar material no verificable. Cubierta inicialmente por el autor.	Contenedores *rootless*, didáctica y documentación reproducible.	P3
Instrucción	Guiar cohortes o un caso real dentro de un alcance y criterio de éxito pactados. Autor; apoyo invitado sólo si se acuerda.	Facilitación, diagnóstico técnico y comunicación con la contraparte.	P4
Revisión externa	Revisar cambios antes de publicar material sensible o cerrar un diagnóstico. No se presupone disponibilidad permanente.	Competencia específica en el componente revisado e independencia de la autoría.	P2–P4

Tabla 21: Funciones operativas, perfiles requeridos y vínculo con P1–P5.

Antes de ampliar P4, la función crítica que falta formalizar es la revisión independiente de entregables para terceros. No se la oculta bajo la denominación de “staff gerencial”: se buscaría una persona con experiencia demostrable en el dominio del piloto, capacidad de reproducir las pruebas y ausencia de conflicto con la entrega, con participación definida por proyecto. También falta resolver el soporte administrativo/legal del primer contrato; no se le atribuye a la dirección académica.

7.4 Estructura al lanzamiento y evolución

La estructura cambia con la evidencia y la demanda, no por crecimiento supuesto. Se distinguen tres momentos compatibles con el cronograma (Sección 5):

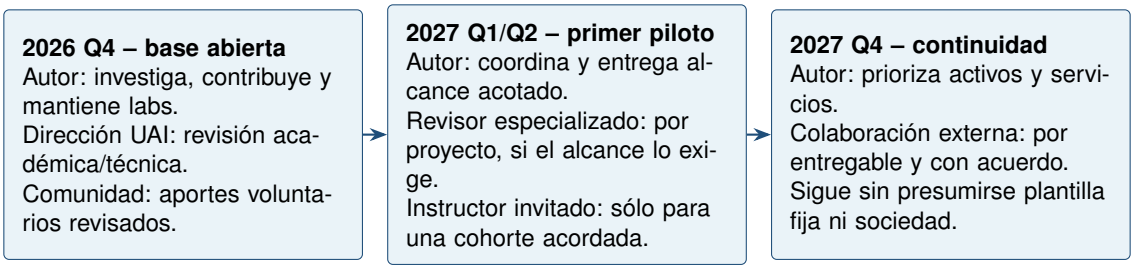


Figura 6: Evolución prevista de funciones; las participaciones futuras son condicionales.

El lanzamiento operativo de P3 ya tiene una plataforma en línea; el “lanzamiento” de esta figura significa ordenar su mantenimiento y los procesos de contribución. El primer servicio pago no aumenta necesariamente la estructura permanente: se aceptará como diagnóstico,

taller o piloto de hasta cuatro semanas, límite fijado para P4 en Sección 5. Para pasar de una etapa a otra deben existir al menos una contraparte, un alcance escrito, capacidad reservada y un criterio de aceptación verificable.

7.5 Filosofía y sistema de trabajo

Rigen los valores ya declarados en el punto 1: rigor, apertura, trazabilidad y reproducibilidad. Se traducen en una secuencia visible de trabajo:

1. **Seleccionar y delimitar.** Registrar el problema o necesidad en un *issue* o ficha; anotar versión, ambiente, alcance y criterio de aceptación. Para un servicio, acordar también tiempo máximo y datos que pueden tratarse.
2. **Reproducir y diseñar.** Seguir el marco de siete fases de la investigación sobre contribuciones: incorporación, selección del problema, reproducción, diseño, implementación, *pull request* y reflexión. Conservar pasos repetibles y una hipótesis comprobable.
3. **Implementar y revisar.** Cambios pequeños, pruebas automáticas cuando corresponda, revisión por pares o por mantenedores *upstream*, y registro del resultado. Un parche enviado no se declara aceptado hasta que exista confirmación externa. En proyectos que lo exijan se respetan DCO, normas de contribución y políticas sobre LLMs.
4. **Publicar y transferir.** Versionar guías/labs, fijar dependencias y fecha de verificación; realizar la cohorte o el piloto con criterios de salida escritos; publicar sólo la evidencia que pueda hacerse pública.
5. **Aprender y mantener.** Recoger devolución, incidencias y resultados de pruebas; convertirlos en trabajo nuevo. El material se actualiza *just in time* al confirmar una cohorte, como se desarrolla en el punto 6.7.6, y se retira si ya no puede verificarse.

El flujo normal es *issue* → reproducción → propuesta → PR → revisión/CI → publicación → devolución. La decisión de publicar un laboratorio o aceptar un servicio corresponde al autor dentro del alcance acordado; la evaluación académica y la aceptación *upstream* pertenecen a sus respectivos revisores. Esta separación evita confundir control editorial con aprobación técnica externa.

7.6 Requerimientos de recursos humanos por etapa

No se propone una nómina fija. La Tabla 22 es una estimación de *capacidad máxima a reservar* para evaluar si los hitos son realizables; las horas del autor fueron confirmadas por él (16/10/2026) como topes que puede sostener. Las horas externas son una posibilidad de contratación o colaboración, no una asignación a las personas nombradas en la Tabla 20.

Etapa	Función / origen	Dedicación prevista	Condición de activación
2026 Q4	Autor: P1–P3 y seguimiento de P5	Hasta 8 h/sem.	Priorizar mantenimiento de activos existentes y entrega académica.
2026 Q4	Comunidad: contribuciones y revisión	0 h comprometidas	Aportes voluntarios sólo si llegan y pasan revisión.
2027 Q1/Q2	Autor: P1–P4	Hasta 12 h/sem.	Primer piloto de hasta cuatro semanas, con alcance y calendario aceptados.
2027 Q1/Q2	Revisor/instructor externo	0–4 h/sem. por piloto	Sólo mediante acuerdo previo y disponibilidad comprobada.
2027 Q4	Autor: continuidad P1–P5	Hasta 16 h/sem.	Aumentar capacidad sólo si los pilotos documentan demanda y financiación.
2027 Q4	Colaborador externo por entregable	0–8 h/sem. por encargo	Contratación o colaboración específica; sin puesto fijo supuesto.

Tabla 22: Plantilla funcional y topes de capacidad: horas del autor confirmadas; horas externas a validar antes de contratar.

El cuello de botella inicial es el tiempo del autor: un piloto no se sumará si desplaza el mantenimiento de parches o la verificación de laboratorios. Antes de aceptar cada encargo P4 se estimarán las horas por paquete de la Tabla 15, se reservará una ventana de revisión y se explicitará quién cubre la función independiente. Si no hay capacidad o forma de contratación resuelta, la actividad se pospone. Esta regla mantiene coherencia entre la estructura, la distribución selectiva del punto 6.7 y el límite de cuatro semanas del primer año.

10. Aspectos Descriptivos de la Solución Tecnológica

10.1 Carátula y datos descriptivos

Datos descriptivos del proyecto:

Ítem	Detalle
Nombre del proyecto	Línea de I+D+i y vinculación tecnológica sobre Go, Software Libre y Podman.
Componente técnico principal	Sitio web catálogo + repositorio de materiales + bot de pedido de servicios.
Autor	Valentín Torassa Colombero.
Legajo	B00069076-T1.
Carrera	Ingeniería en Sistemas Informáticos.
Facultad / Sede	Facultad de Tecnología Informática – UAI Rosario.
Asignatura	Seminario de Trabajo Final.
Período	Segunda Entrega – segundo cuatrimestre 2026.

10.2 Índice

Véase tabla de contenidos al inicio del documento.

10.3 Historial de Revisión

Versión	Responsable	Descripción de la Revisión	Fecha
0.1	V. Torassa Colombero	Esquema inicial del componente técnico.	2026-05-03
0.5	V. Torassa Colombero	Casos de uso y diagramas preliminares.	2026-05-10
1.0	V. Torassa Colombero	Primera versión completa para entrega.	2026-05-17

10.4 Documento Visión

10.4.1 Propuesta de valor del proyecto

La “solución tecnológica” de este Trabajo Final no es una aplicación desarrollada desde cero. Es un **marco metodológico reproducible** para contribuir a un proyecto de software libre maduro – Podman – acompañado de un **kit de laboratorios** para evaluar funcionalidades rootless y de **contribuciones reales verificables al repositorio upstream** `containers/podman`. La integración de estos tres componentes constituye el aporte de Ingeniería de Software de la línea.

- **Marco metodológico:** las siete fases definidas en el primer borrador CACIC [7] – Onboarding, Selección, Reproducción, Diseño, Implementación, Pull Request y Reflexión – con artefactos verificables en cada paso.
- **Kit de laboratorios:** los cuatro escenarios del protocolo de evaluación definido en el segundo borrador CACIC [6] (contenedor simple, servicio web rootless, pod multi-contenedor, servicio persistente con Quadlet).
- **Contribuciones reales:** pull requests aceptados o en revisión en `containers/podman`. Caso

testigo: PR #28637 “Add `-retry` support to manifest push” [5], abierto el 3 de mayo de 2026 y **mergeado el 23 de junio de 2026**.

Esta propuesta beneficia simultáneamente a tres públicos: el estudiante o profesional que aplica el marco (gana evidencia profesional verificable), la comunidad Podman (recibe contribuciones de calidad documentada) y la cátedra (obtiene un trabajo aplicado con resultados externamente auditables).

10.4.2 Especificaciones de requerimientos

Los requerimientos se expresan sobre el marco metodológico, no sobre un producto de software propio.

Requerimientos funcionales:

- RF-01: cada contribución producida bajo el marco debe estar asociada a un issue triado del repositorio o, en su defecto, a una reproducción documentada.
- RF-02: cada fase del marco debe producir al menos un artefacto consultable (ficha de onboarding, matriz de selección, script de reproducción, mini diseño, PR, bitácora de revisión, ficha de reflexión).
- RF-03: cada laboratorio publicado debe declarar la versión de Podman testeada (`podman info`), distribución de prueba y modo root/rootless.
- RF-04: cada pull request al repositorio upstream debe incluir referencia al issue (`Fixes: #N`), descripción del cambio en palabras propias, pruebas asociadas y, si corresponde, actualización de documentación y Swagger.
- RF-05: las contribuciones deben mantener trazabilidad de la versión de Podman analizada en los papers (rama `main`, commit `9f02e222`, 2026-05-01).

Requerimientos no funcionales:

- RNF-01: cumplimiento estricto de `CONTRIBUTING.md` de Podman: commits firmados con DCO (`git commit -s`), formato `gofumpt`, validación `make validatepr` en verde antes de enviar el PR.
- RNF-02: cumplimiento de `LLM_POLICY.md` [4]: ningún texto generado por LLM se publica verbatim en issues, PRs, commits ni comunicaciones con mantenedores.
- RNF-03: los laboratorios deben ejecutarse en un host Linux estándar con cgroups v2 y user namespaces habilitados, sin requerir hardware especializado.
- RNF-04: la documentación de cada contribución debe permitir reproducción independiente por un tercero en menos de 30 minutos.
- RNF-05: el material producido se publica con licencia CC-BY-SA 4.0; las contribuciones de código respetan la licencia Apache 2.0 del proyecto.

10.4.3 Alcance del desarrollo

- **Dentro del alcance:**
  - Marco metodológico de siete fases documentado y aplicado.
  - Kit con los cuatro laboratorios de los escenarios del protocolo de evaluación (contenedor simple; servicio web rootless; pod multi-contenedor; servicio persistente con Quadlet).
  - Bitácora de contribuciones efectivas al repositorio `containers/podman`: caso testigo PR #28637 (en revisión) y siguientes PRs planificados en áreas de documentación, pruebas de regresión, mensajes de error y compatibilidad CLI.
  - Fichas de trazabilidad académica por contribución.
- **Fuera del alcance:**
  - Desarrollo de un sistema propio desde cero, sea SaaS, LMS, CRM, plataforma de pagos o aplicación independiente. La línea se apoya sobre el código de Podman y herramientas existentes.
  - Modificaciones profundas a `libpod` (núcleo de ciclo de vida) reservadas para etapas avanzadas con dominio consolidado.

- Operación de orquestadores cluster (Kubernetes/OpenShift), salvo como referencia comparativa para Quadlet/Pods.

10.4.4 Definiciones, acrónimos y abreviaciones

- **Podman:** motor de gestión de contenedores OCI sin daemon, con soporte rootless [3].
- **Rootless:** ejecución de contenedores con usuario sin privilegios; root del contenedor mapeado a un usuario no privilegiado del host mediante user namespaces.
- **OCI:** Open Container Initiative; especificaciones de imagen y runtime [1, 2].
- **Pod:** agrupación de contenedores que comparten recursos (red, IPC).
- **Quadlet:** mecanismo declarativo para describir unidades systemd que orquestan contenedores Podman.
- **Netavark** / **Aardvark-DNS** / **pasta:** stack de networking del ecosistema containers.
- **crun** / **runc:** runtimes OCI; `crun` es el predeterminado en Podman moderno.
- **Buildah:** herramienta de construcción de imágenes OCI del mismo ecosistema.
- **cgroups (v1/v2):** mecanismo del kernel para limitar y contabilizar recursos.
- **subuid/subgid:** archivos del sistema que definen rangos de UIDs/GIDs subordinados para user namespaces.
- **DCO:** Developer Certificate of Origin; firma obligatoria en cada commit del proyecto.
- **Cirrus CI:** sistema de integración continua usado por Podman.
- **`make validatepr`:** target del Makefile que ejecuta formato, lint y validaciones requeridas por el proyecto antes del PR.
- **Manifest list:** índice OCI que apunta a múltiples imágenes (multi-arquitectura); operado mediante `podman manifest`.
- **Vendoring:** práctica de copiar dependencias dentro del repositorio (directorio `vendor/` en Podman); no se edita manualmente.

10.4.5 Descripción del entorno del sistema

- **Entorno de desarrollo del contribuyente (lado "cliente"):** workstation Linux (Fedora 41/42 o Ubuntu 24.04 LTS recomendadas), Go $\geq$ 1.22, Podman compilado desde fuente, `golangci-lint`, Ginkgo para integración, BATS para system tests, Git, herramientas estándar de Unix.
- **Entorno de pruebas locales:** kernel reciente con cgroups v2 habilitado, user namespaces activos, subuids/subgids configurados, runtime `crun` disponible.
- **Entorno upstream (lado "servidor" de la colaboración):** GitHub (`github.com/containers/podman`), Cirrus CI con matriz de validación por sistema operativo, mantenedores y revisores del proyecto.
- **Políticas de seguridad y operación:** ejecución rootless por defecto durante el desarrollo, secrets de CI gestionados por Cirrus, integridad de commits por firma DCO, ausencia de credenciales personales en repositorios.
- **Políticas de backup:** el repositorio Git es la fuente de verdad; clones espejados en GitHub personal del contribuyente y respaldo local del workstation.

10.5 Descripción Global del Producto

10.5.1 Mapa del marco metodológico de siete fases

El equivalente al "mapa de navegación" es el flujo de las siete fases que recorre cada contribución (Figura 7). Una contribución inicia en Onboarding y termina en Reflexión; la fase de PR admite ciclos de revisión con la comunidad.

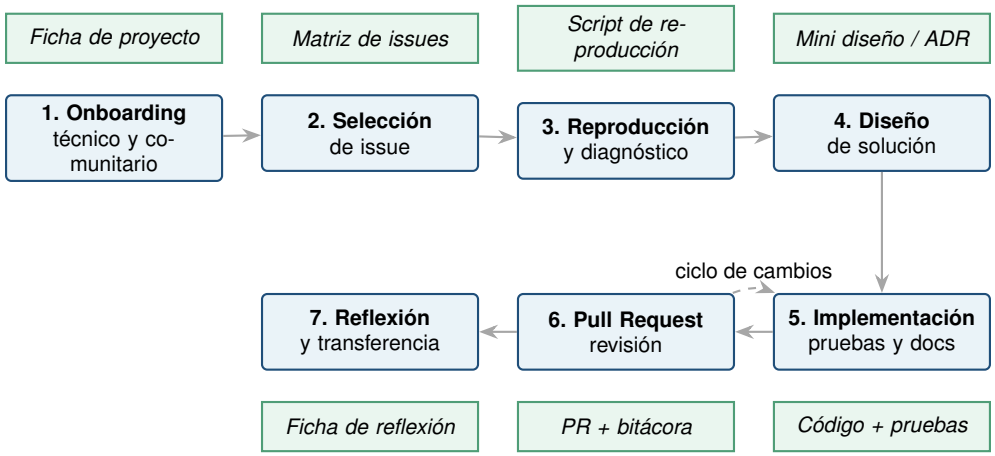


Figura 7: Mapa de las siete fases del marco metodológico, con sus artefactos verificables. Adaptado de [7].

10.5.2 Índice de casos de uso

- CU-001-001: Onboarding al ecosistema Podman (Fase 1).
- CU-001-002: Selección y priorización de un issue para contribuir (Fase 2).
- CU-001-003: Reproducción mínima de un defecto o feature gap (Fase 3).
- CU-001-004: Implementación y envío de un Pull Request al proyecto Podman (Fases 4 a 6).
- CU-001-005: Ejecución de un escenario del protocolo de evaluación (laboratorio rootless / pods / Quadlet).
- CU-001-006: Reproducción de un laboratorio publicado por un tercero (consumo del kit).

10.5.3 Casos de Uso

Cada caso de uso se acompaña de su propio diagrama, con los actores que intervienen y las relaciones de inclusión o extensión con los demás casos. El orden temporal de las fases no se representa en estos diagramas: está en el mapa de siete fases (Figura 7).

CU-001-001 Onboarding al ecosistema Podman

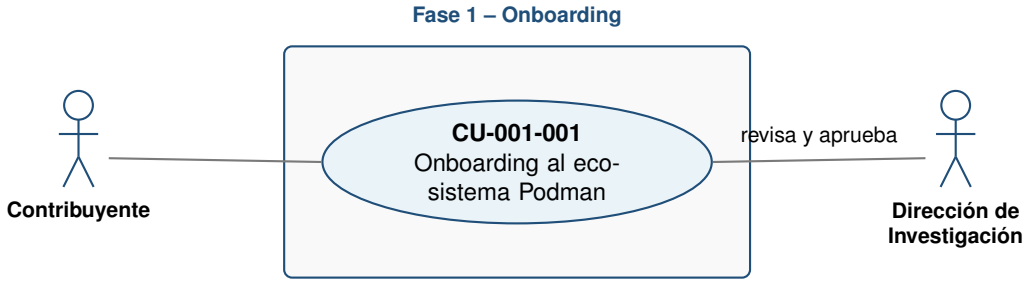


Figura 8: CU-001-001 – Onboarding al ecosistema Podman.

- **Objetivo:** comprender el proyecto antes de modificarlo y producir una ficha de onboarding.
- **Actores primarios:** Contribuyente.
- **Actores secundarios:** Dirección de Investigación (revisa la ficha).
- **Pre-condición:** workstation con Linux, Git y Go instalados.
- **Post-condición:** ficha de onboarding completada y aprobada por dirección.
- **Escenario principal de éxito:**
  1. El contribuyente clona el repositorio `containers/podman` en la rama `main`.
  2. Lee `README.md`, `CONTRIBUTING.md`, `LLM_POLICY.md` y `docs/CODE_STRUCTURE.md`.

3. Ejecuta `make binaries` y `make validatepr` para confirmar entorno funcional.
4. Construye la ficha de onboarding (dominio, tecnologías, licencia, canales, mapa de directorios, comandos de build/test).
5. La Dirección de Investigación revisa la ficha y aprueba.

- **Caminos alternativos:** (a) build falla por dependencias → instalar y volver al paso 3; (b) la ficha omite reglas de comunicación → se completa y se vuelve al paso 5.

CU-001-002 Selección y priorización de un issue

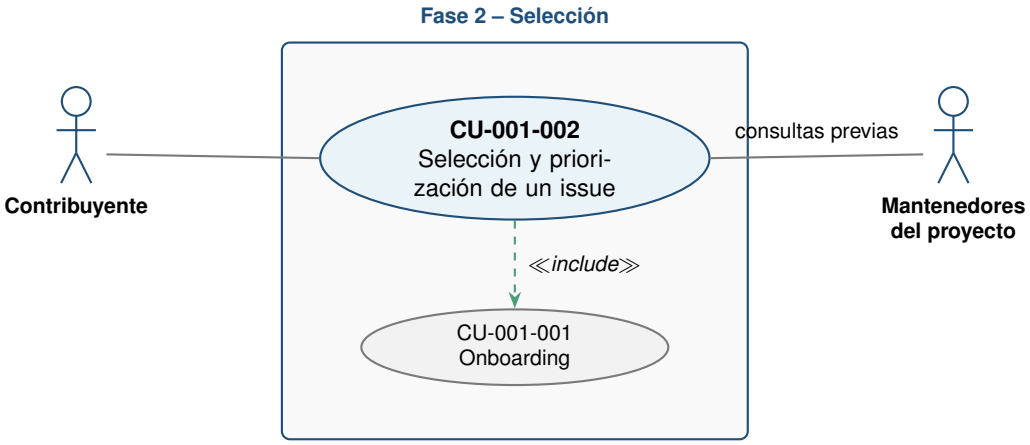


Figura 9: CU-001-002 – Selección y priorización de un issue para contribuir.

- **Objetivo:** elegir un issue adecuado para una primera contribución, aplicando los cinco criterios del marco (reproducibilidad, alcance, riesgo, alineación, valor).
- **Actores primarios:** Contribuyente.
- **Actores secundarios:** Mantenedores del proyecto (responden consultas previas si las hubiera).
- **Pre-condición:** ficha de onboarding aprobada.
- **Post-condición:** issue seleccionado con justificación documentada en la matriz de selección.
- **Escenario principal de éxito:**
  1. El contribuyente filtra issues por etiquetas como `Good First Issue` y `triaged`.
  2. Evalúa cinco issues candidatos en la matriz según los cinco criterios.
  3. Selecciona el issue con mayor puntaje agregado y menor riesgo.
  4. Documenta la selección y publica un comentario (`/assign` si corresponde).
- **Ejemplo (caso testigo):** se seleccionó el issue #28590 “Podman-manifest-push have no retry option” [9], etiquetado `Good First Issue` y `triaged`, por su paridad clara con `podman push -retry` y alcance acotado.

CU-001-003 Reproducción mínima de un defecto o feature gap

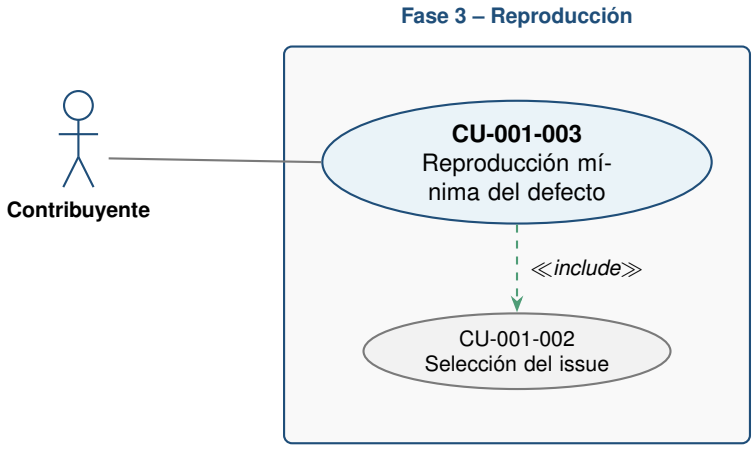


Figura 10: CU-001-003 – Reproducción mínima de un defecto o *feature gap*. Es el único caso sin actores secundarios: se resuelve por completo en el entorno del contribuyente.

- **Objetivo:** producir evidencia reproducible que confirme el estado actual y permita validar el cambio posterior.
- **Actores primarios:** Contribuyente.
- **Pre-condición:** issue seleccionado.
- **Post-condición:** script de reproducción que demuestra la diferencia entre comportamiento esperado y obtenido.
- **Escenario principal de éxito:**
  1. Registrar entorno: `podman info`, versión, distribución, kernel, modo root/rootless.
  2. Construir comandos mínimos que evidencien el feature gap o el bug.
  3. Capturar salida esperada (de la documentación o de comandos equivalentes) y salida obtenida.
  4. Guardar el script en `labs/issue-<N>/repro.sh`.
- **Caso testigo:** para #28590, la reproducción consistió en demostrar que `podman push -retry` acepta los flags mientras que `podman manifest push -retry` no los reconoce.

CU-001-004 Implementación y envío de un Pull Request

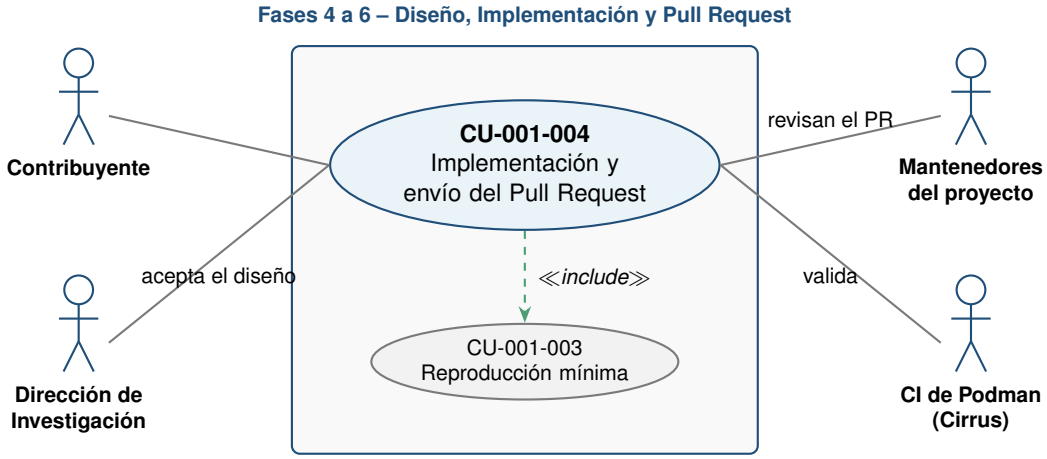


Figura 11: CU-001-004 – Implementación y envío de un Pull Request al proyecto Podman.

- **Objetivo:** implementar el cambio, validarlo localmente y abrir el PR upstream.
- **Actores primarios:** Contribuyente.
- **Actores secundarios:** CI de Podman (Cirrus), mantenedores (revisión).
- **Pre-condición:** reproducción documentada y diseño aceptado por la Dirección de Investigación.
- **Post-condición:** PR abierto en `containers/podman` con CI verde inicial.
- **Escenario principal de éxito:**
  1. Crear rama y aplicar cambios respetando los módulos del repositorio (`cmd/podman`, `pkg/api`, `pkg/bindings`, `pkg/domain`, `docs`).
  2. Actualizar pruebas (e2e con Ginkgo y/o BATS) y documentación (manpages y Swagger si toca API).
  3. Ejecutar `make validatepr` y resolver hallazgos.
  4. Hacer commit firmado (`git commit -s`) y push a la rama personal.
  5. Abrir el PR contra `containers/podman:main` con descripción en palabras propias y `Fixes: #N`.
  6. Atender el ciclo de revisión de la comunidad.
- **Caso testigo:** PR #28637 “Add `-retry` support to manifest push” [5]; 87 líneas agregadas, 10 eliminadas, 9 archivos modificados, etiqueta `kind/api-change`; cubre CLI, REST API, bindings remotos, manpages, Swagger y pruebas e2e.

CU-001-005 Ejecución de un escenario del protocolo de evaluación

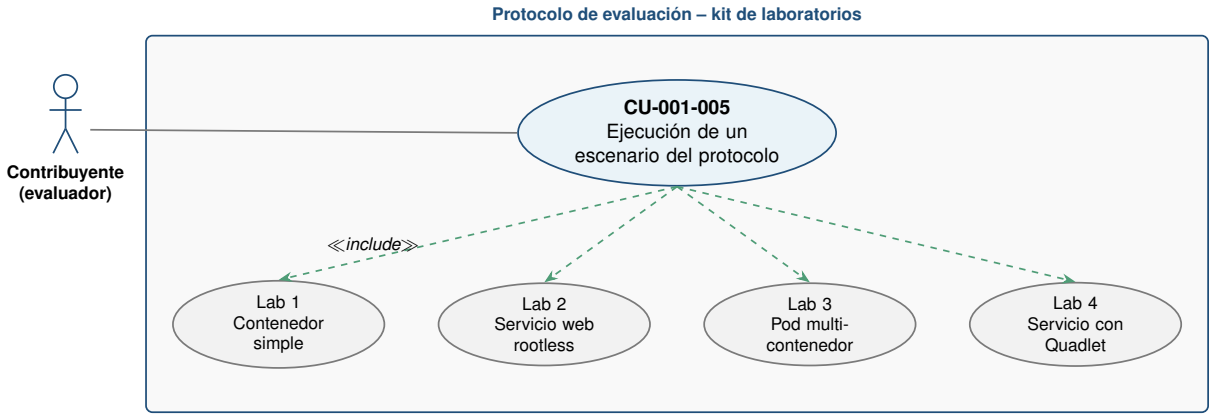


Figura 12: CU-001-005 – Ejecución de un escenario del protocolo de evaluación. Los cuatro laboratorios del kit son casos incluidos.

- **Objetivo:** validar empíricamente una capacidad de Podman y producir un laboratorio reproducible.
- **Actores primarios:** Contribuyente (con rol de evaluador).
- **Pre-condición:** workstation con Podman instalado y configurado.
- **Post-condición:** laboratorio publicado con script, registro de comandos, salidas y observaciones convertibles en issue, prueba o documentación.
- **Escenarios cubiertos (del paper de análisis funcional [6]):**
  1. Lab 1 – Contenedor simple: imagen mínima, volumen, variables de entorno, logs.
  2. Lab 2 – Servicio web rootless: publicar puerto no privilegiado y validar conectividad.
  3. Lab 3 – Pod multi-contenedor: servicio principal y sidecar con red compartida.
  4. Lab 4 – Servicio persistente con Quadlet: unidad systemd, arranque, reinicio y logs.

CU-001-006 Reproducción de un laboratorio publicado por un tercero

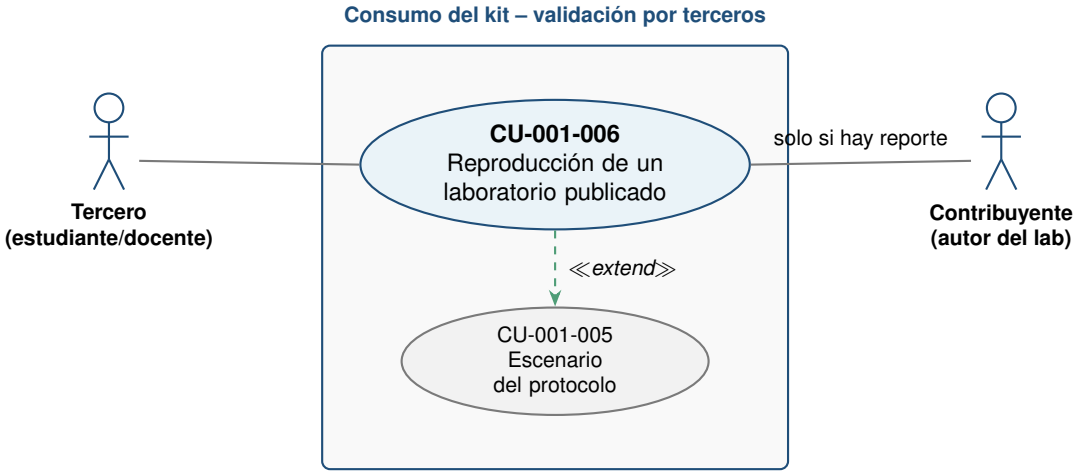


Figura 13: CU-001-006 – Reproducción de un laboratorio publicado por un tercero.

- **Objetivo:** verificar que una persona ajena al proyecto puede reproducir un laboratorio publicado sin asistencia del autor.
- **Actores primarios:** Tercero (estudiante, docente o profesional).

- **Actores secundarios:** Contribuyente (autor del laboratorio, solo si el tercero reporta un problema).
- **Pre-condición:** laboratorio publicado en Open Security Labs, con entorno y versiones declarados.
- **Post-condición:** laboratorio reproducido en menos de 30 minutos (RNF-04), o reporte de la desviación encontrada.
- **Escenario principal de éxito:**
  1. El tercero accede al laboratorio publicado, sin registro ni credenciales.
  2. Verifica los requisitos declarados: versión de Podman, distribución y modo root o *rootless*.
  3. Ejecuta los comandos del laboratorio en su propio entorno.
  4. Compara la salida obtenida con la salida esperada que documenta el laboratorio.
  5. Registra el resultado y, si corresponde, abre un *issue* o *pull request* en el repositorio.
- **Caminos alternativos:** (a) la salida difiere por versión de Podman $\rightarrow$ se documenta la diferencia y se actualiza la matriz de compatibilidad del laboratorio; (b) el entorno no cumple los requisitos (*cgroups* v1, `subuid` sin configurar) $\rightarrow$ el laboratorio deriva a la guía de preparación de entorno.
- **Evidencia:** tres contribuyentes externos ya reprodujeron laboratorios y aportaron *pull requests* mergeados (serie #22–#28), lo que valida este caso de uso empíricamente.

10.5.4 Diagrama de secuencia

Para ilustrar cómo las tres líneas de investigación convergen sobre el código real, la Figura 14 muestra el **flujo interno de** `podman manifest push --retry`: el camino que recorre el comando desde la CLI hasta el *registry* OCI. El caso testigo PR #28637 introduce los *flags* `--retry` y `--retry-delay` en este camino, en paridad con `podman push`.

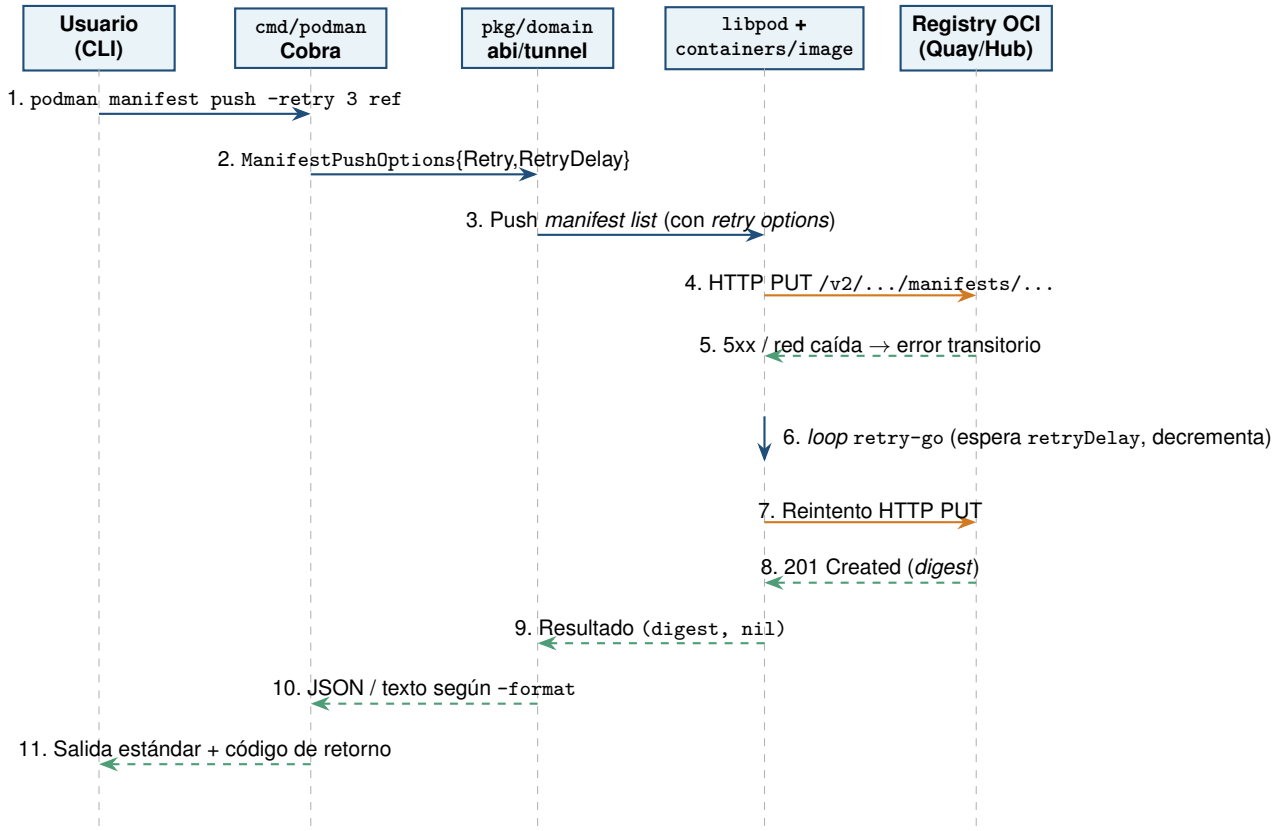
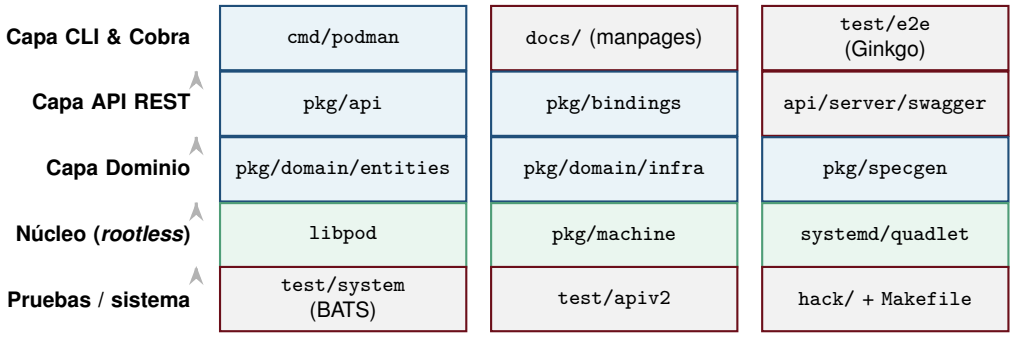


Figura 14: Diagrama de secuencia del comando `podman manifest push -retry`. Atraviesa CLI (Cobra), capa de dominio, núcleo `libpod/containers/image` y *registry* OCI. Caso testigo del flujo: PR #28637.

10.5.5 Diagrama de paquetes

El objeto de estudio principal es el repositorio `containers/podman`. La Figura 15 muestra sus paquetes Go más relevantes, agrupados por capa, e indica con bordes de colores qué líneas de investigación tocan cada zona.



Azul paquetes con patrones *Go-idiomáticos* relevantes para [8]: `cmd/podman` (Cobra, errores como valores), `pkg/api` (*handlers*), `pkg/bindings` (cliente REST tipado), `pkg/domain` (interfaces ABI/Tunnel intercambiables), `pkg/specgen` (validación por tipos).

Verde paquetes que materializan las abstracciones funcionales descritas en [6]: `libpod` (contenedores, pods, runtime), `pkg/machine` (VMs Mac/Win), `pkg/systemd/quadlet` (unidades declarativas).

Rojo superficies de contribución con bajo riesgo de regresión identificadas en [7]: `docs/`, Swagger, suites `test/e2e`, `test/system` y `test/apiv2`; donde aterrizó el PR #28637.

Figura 15: Diagrama de paquetes de `containers/podman` agrupado por capa. La flecha lateral indica el flujo principal de invocación CLI → API → Dominio → Núcleo → Pruebas.

10.5.6 Diagrama de componentes

El sistema sobre el que se trabaja es el *ecosistema Podman*; el contribuyente es el componente externo que interactúa con él. La Figura 16 muestra los principales componentes y dependencias del ecosistema, basadas en el análisis del repositorio sobre commit `9f02e222`.

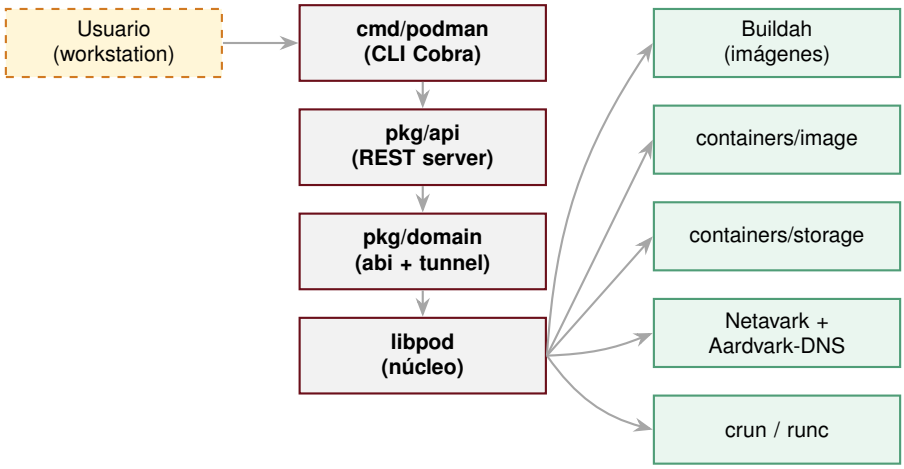


Figura 16: Componentes principales del ecosistema Podman: cadena interna (CLI → API → dominio → libpod) y dependencias del núcleo sobre el resto del proyecto `containers/`. Adaptado de [6, 3].

10.5.7 Diagrama de Clases (modelo de dominio de Podman)

Las entidades centrales del dominio Podman, tal como se modelan en `libpod` y en `pkg/specgen`, se muestran en la Figura 17. Es el dominio que las tres líneas de investigación estudian (idiomas Go en [8]), aplican (contribuciones en [7]) y describen (análisis funcional en [6]).

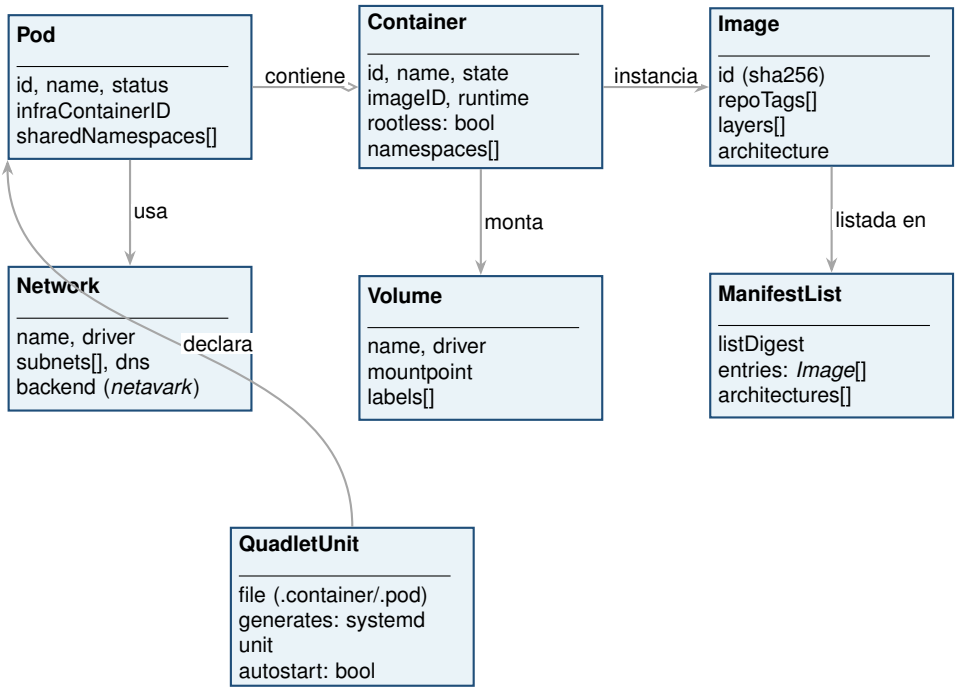


Figura 17: Modelo de dominio de Podman: `Pod`, `Container`, `Image`, `ManifestList`, `Network`, `Volume` y `QuadletUnit`, con sus relaciones principales. Tomado del análisis sobre commit `9f02e222`.

10.5.8 Diagrama de Entidad Relación

Podman no usa una base relacional, pero `containers/storage` y `libpod` persisten objetos con relaciones claras (BoltDB / SQLite según *backend*). La Figura 18 expresa ese estado como esquema relacional equivalente, útil para razonar sobre integridad y trazabilidad en los laboratorios del kit.

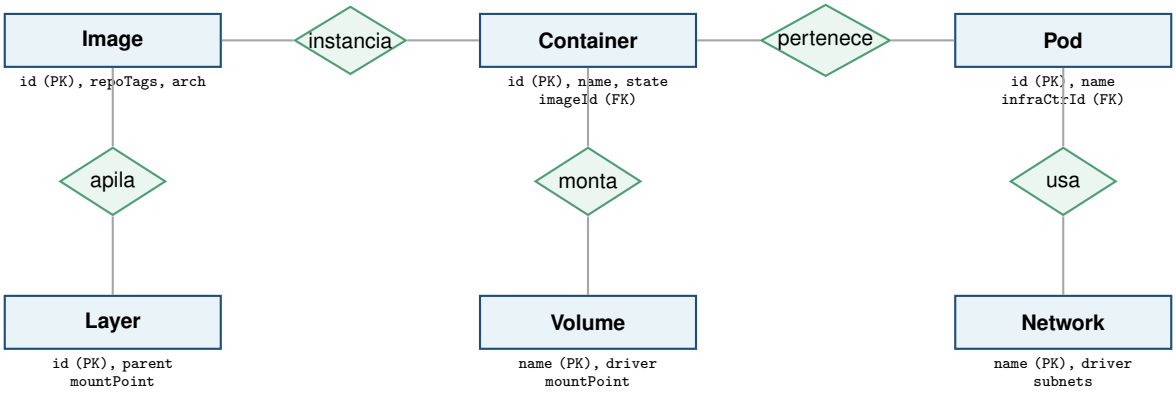


Figura 18: Esquema relacional equivalente al estado persistido por `libpod` y `containers/storage`.

Diccionario de datos:

- `Image(id PK, repoTags, architecture).`
- `Layer(id PK, parent FK → Layer, mountPoint, size).`
- `Container(id PK, name, state, imageId FK → Image, runtime, rootless).`
- `Pod(id PK, name, infraContainerId FK → Container, sharedNamespaces).`
- `Volume(name PK, driver, mountPoint, labels).`

- `Network(name PK, driver, subnets, dns, backend)`.

10.5.9 Diagrama de despliegue

La Figura 19 muestra el despliegue de Podman en modo *rootless* sobre un host Linux. Es la topología analizada en detalle por [6] y la que se reproduce en los cuatro escenarios del protocolo de evaluación (Lab 1 a Lab 4). El *registry* OCI aparece como nodo externo de borde.

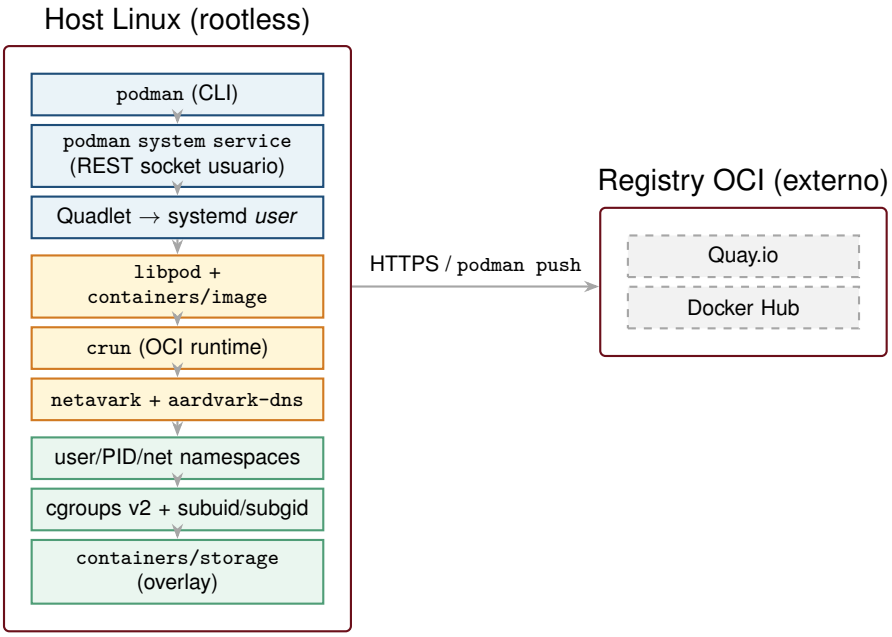


Figura 19: Despliegue de Podman *rootless* sobre un host Linux: *userspace* (CLI, *service*, Quadlet), runtime (`libpod`, `crun`, networking) y soporte del kernel (namespaces, cgroups v2, storage). Caso testigo PR #28637 ejerce la flecha hacia el *registry*.

10.5.10 Especificación de Casos de Prueba (extracto)

- **CP-001 (validación local):** `make validatepr` ejecuta formato, lint y validaciones → retorna 0 antes de abrir el PR.
- **CP-002 (DCO):** cada commit lleva firma `Signed-off-by:` → verificador DCO del PR pasa en verde.
- **CP-003 (CI integration):** pipeline Cirrus ejecuta integration tests (Ginkgo) y unit tests aplicables al cambio → verde.
- **CP-004 (CI system):** BATS suite cubre escenarios CLI relevantes al cambio (caso PR #28637: `podman manifest push -retry` y `-retry-delay`) → verde.
- **CP-005 (paridad funcional, caso PR #28637):** comparar comportamiento de `podman push -retry` vs. `podman manifest push -retry` en escenarios de fallo simulado → ambos reintentan idénticamente.
- **CP-006 (paridad REST API, caso PR #28637):** request a la REST API con parámetros `retry` y `retryDelay` → comportamiento documentado en Swagger y respuesta consistente.
- **CP-007 (laboratorio rootless):** Lab 2 ejecuta un servicio web en puerto $>$ 1024 sin privilegios y un `curl` desde el host responde 200.
- **CP-008 (laboratorio Quadlet):** Lab 4 crea una unidad systemd; `systemctl -user restart` reinicia el contenedor y los logs persisten vía journald.
- **CP-009 (reproducción $\leq$ 10 comandos):** la reproducción del issue seleccionado se ejecuta en $\leq$ 10 comandos documentados sobre un host limpio.

10.6 Trazabilidad de requerimientos

10.6.1 Criterios de aceptación

Cada requerimiento funcional declara al menos un criterio de aceptación comprobable, expresado como resultado observable y no como tarea a realizar.

Req.	Criterio de aceptación	Cómo se comprueba
RF-01	Toda contribución registrada expone su identificador upstream y su estado actual.	Listado de contribuciones contra la API del repositorio; no debe haber filas sin identificador.
RF-02	Cada una de las siete fases tiene al menos un artefacto consultable asociado.	Recorrido del índice de artefactos: siete entradas no vacías.
RF-03	Ningún laboratorio publicado carece de versión declarada de Podman y del sistema base.	Validador de metadatos sobre el conjunto de laboratorios: cero incumplimientos.
RF-04	Todo *pull request* enviado referencia el *issue* que lo origina.	Inspección del cuerpo del PR; la referencia debe resolver a un issue existente.
RF-05	Dada una contribución, se puede reconstruir la versión exacta sobre la que se trabajó.	Resolución del *commit* base declarado en el repositorio upstream.

Tabla 23: Criterios de aceptación por requerimiento funcional.

10.6.2 Matriz de trazabilidad requerimiento → caso de uso

La matriz permite recorrer cada requerimiento hasta el caso de uso que lo realiza y, en sentido inverso, verificar que ningún caso de uso existe sin requerimiento que lo motive.

	CU-001	CU-002	CU-003	CU-004	CU-005	CU-006
RF-01	–	●	–	●	–	–
RF-02	●	●	●	●	●	–
RF-03	–	–	–	–	●	●
RF-04	–	–	–	●	–	–
RF-05	–	–	●	●	–	●
RNF-01	●	–	–	●	–	–
RNF-02	–	–	–	●	–	–
RNF-03	–	–	●	–	●	●
RNF-04	●	●	●	●	●	●
RNF-05	–	–	–	–	●	●

Tabla 24: Matriz de trazabilidad. Cada columna abrevia `CU-001-00n`.

Se verifican las dos direcciones: toda fila tiene al menos una marca (no hay requerimiento huérfano) y toda columna tiene al menos una marca (no hay caso de uso sin requerimiento que lo justifique).

10.7 Diagrama general de casos de uso

El diagrama general ubica los seis casos dentro del límite del sistema y separa los actores primarios de los secundarios.

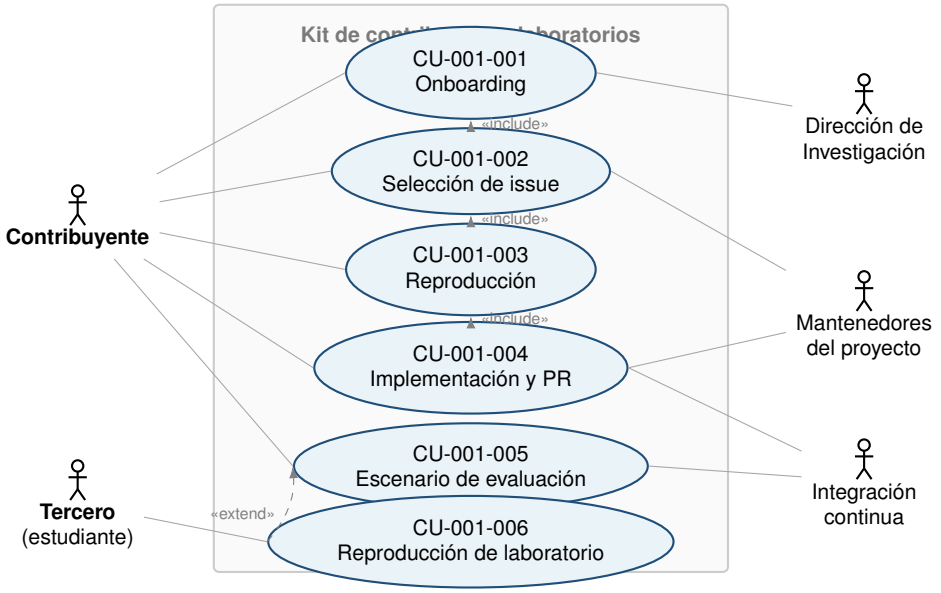


Figura 20: Diagrama general de casos de uso con límite del sistema, actores primarios (izquierda) y secundarios (derecha).

10.8 Cursos básicos, alternativos y excepciones

Cada caso se especifica con su curso básico en voz activa, sus cursos alternativos y las excepciones previstas. Se omite por brevedad la repetición de precondiciones ya declaradas en la Sección 10.6.

CU-001-001 — Onboarding al ecosistema Podman	
Curso básico	El contribuyente clona el repositorio, ejecuta la compilación local, registra la versión y el *commit* base, y completa la ficha de onboarding. La dirección revisa la ficha y la aprueba.
Alternativos	Si la compilación local falla, el contribuyente registra el error y utiliza el entorno contenedorizado provisto por el proyecto.
Excepciones y reglas	**E1:** dependencias del sistema incompatibles → se documenta la desviación y se fija la versión alternativa (RNF-03). **Regla:** ninguna fase avanza sin artefacto consultable (RF-02).

CU-001-002 — Selección y priorización de un issue	
Curso básico	El contribuyente filtra los issues abiertos por etiqueta y antigüedad, descarta los que ya tienen asignado, evalúa el esfuerzo estimado y registra el issue elegido con su justificación.
Alternativos	Si ningún issue del filtro resulta viable, el contribuyente amplia el criterio a documentación o pruebas, que también producen contribución válida.
Excepciones y reglas	**E1:** el issue es reclamado por otra persona durante la evaluación → se descarta y se retoma el filtro. **Regla:** toda consulta previa a los mantenedores queda registrada en el hilo público del issue.

CU-001-003 — Reproducción mínima de un defecto	
Curso básico	El contribuyente construye el caso mínimo que expone el comportamiento, lo ejecuta sobre la versión base declarada, registra la salida obtenida y la esperada, y adjunta la evidencia al issue.
Alternativos	Si el comportamiento no se reproduce, el contribuyente documenta el intento y solicita precisiones en el issue antes de cerrar la fase.
Excepciones y reglas	**E1:** el defecto depende de hardware no disponible → se declara fuera de alcance y se elige otro issue. **Regla:** la reproducción debe ser ejecutable por un tercero sin asistencia (RNF-03).

CU-001-004 — Implementación y envío del Pull Request	
Curso básico	El contribuyente implementa el cambio acotado, agrega o ajusta las pruebas, ejecuta la validación local, firma los *commits*, abre el *pull request* referenciando el issue y atiende la revisión hasta la integración.
Alternativos	Si la revisión solicita un cambio de enfoque, el contribuyente reabre la fase de diseño y repite la validación antes de actualizar el PR.
Excepciones y reglas	**E1:** la validación local falla → el envío se bloquea hasta corregir. **E2:** la integración continua falla en un entorno específico → se reproduce ese entorno antes de insistir. **Regla:** sin firma DCO el PR no se considera enviado (RNF-02).

CU-001-005 — Ejecución de un escenario de evaluación	
Curso básico	El contribuyente selecciona el escenario del protocolo, prepara el entorno *rootless*, ejecuta los pasos, registra las mediciones y publica el resultado junto con la versión utilizada.
Alternativos	Si el escenario requiere privilegios elevados, se ejecuta la variante equivalente documentada y se deja constancia de la diferencia.
Excepciones y reglas	**E1:** el entorno no alcanza los requisitos mínimos → el escenario se marca como no ejecutado, nunca como fallido. **Regla:** toda medición publicada declara la versión del sistema base (RF-03).

CU-001-006 — Reproducción de un laboratorio por un tercero	
Curso básico	El tercero accede al laboratorio publicado, verifica los requisitos declarados, ejecuta las instrucciones en su propio entorno y compara el resultado obtenido con el esperado.
Alternativos	Si el tercero detecta una discrepancia, abre un reporte sobre el laboratorio, que se trata con el mismo circuito que un issue upstream.
Excepciones y reglas	**E1:** versión del sistema base distinta de la declarada → el laboratorio advierte la incompatibilidad antes de ejecutar. **Regla:** el material se distribuye con licencia abierta y sin registro previo (RNF-05).

10.9 Prototipo de interfaz

El prototipo se presenta como **especificación de interacción**, no como producto terminado: fija qué datos y qué controles debe exponer cada pantalla, sin comprometer decisiones visuales definitivas. Su función es permitir que las especificaciones de casos de uso se validen contra algo observable antes de diseñar la solución.

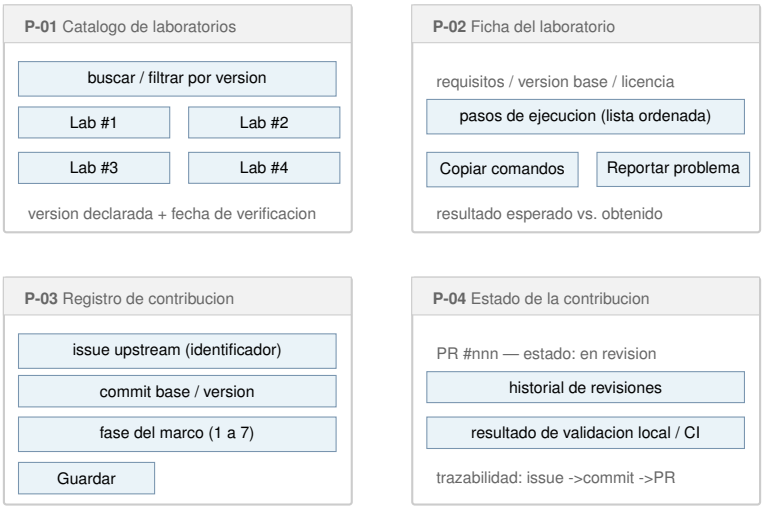


Figura 21: Prototipo de baja fidelidad. Cada pantalla fija datos y controles requeridos, no su apariencia final.

Pant.	Caso de uso	Actor	Datos y controles que expone
P-01	CU-001-006	Tercero	Listado, filtro por versión, fecha de verificación (RF-03).
P-02	CU-001-005, CU-001-006	Tercero, Contribuyente	Requisitos, pasos, resultado esperado, alta de reporte.
P-03	CU-001-001, CU-001-002	Contribuyente	Issue, *commit* base, fase del marco (RF-02, RF-05).
P-04	CU-001-004	Contribuyente, Mantenedores	Estado del PR, revisiones, resultado de validación (RF-01, RF-04).

Tabla 25: Vinculación de cada pantalla con casos de uso, actores y datos especificados.

Correspondencia con las especificaciones. Los controles de la Figura 21 se derivan uno a uno de los cursos básicos de la Sección 10.8: no hay control sin paso que lo requiera, ni paso que exija un dato que la pantalla no exponga. **Revisión exploratoria con usuarios.** El 14/10/2026, una persona del perfil contribuyente probó P-02, P-03 y P-04 y encontró la información necesaria en las tres. El 16/10/2026, un revisor de PR probó P-04 y localizó el estado y el resultado de la validación. El 17/10/2026, un estudiante ajeno a la cursada de Seminario probó P-01 y P-02: encontró un laboratorio para Podman 5.x, su fecha de verificación y cómo reportar un problema. Entre los tres se revisaron las cuatro pantallas. Según el registro del autor, la valoración fue positiva y no se sugirieron cambios. La devolución fue verbal; no se midieron tiempos ni se recogió devolución escrita. Esta revisión inicial no acredita por sí sola la validación con cohortes de cátedra y equipo técnico prevista en O4.

10.10 Diagramas de robustez

Los diagramas de robustez cierran la distancia entre el texto del caso de uso y el diseño. Se dibuja uno por caso, con los cuatro estereotipos de ICONIX y respetando las reglas de comunicación: el actor sólo habla con fronteras; las fronteras sólo con controles; las entidades sólo se alcanzan desde un control; y no hay comunicación directa entre dos fronteras ni entre dos entidades.



Figura 22: Notación empleada en los diagramas de robustez.

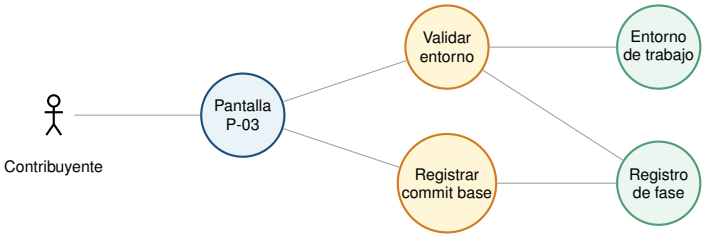


Figura 23: Diagrama de robustez de CU-001-001 — Onboarding.

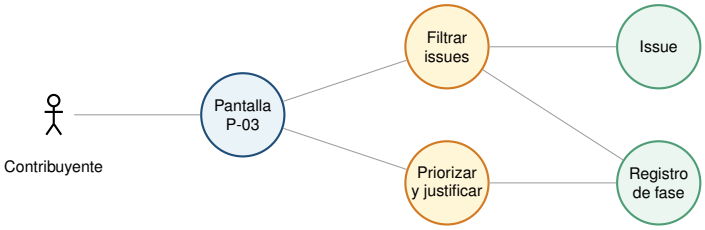


Figura 24: Diagrama de robustez de CU-001-002 — Selección de issue.

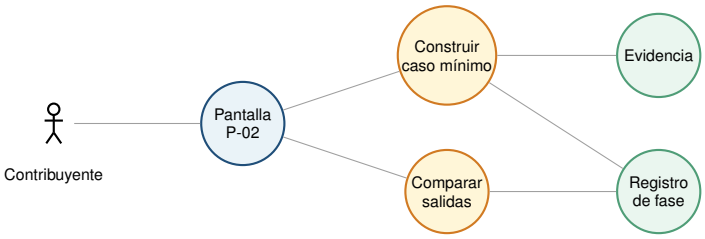


Figura 25: Diagrama de robustez de CU-001-003 — Reproducción.

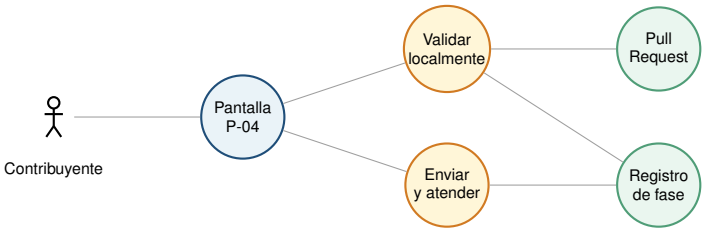


Figura 26: Diagrama de robustez de CU-001-004 — Implementación y PR.

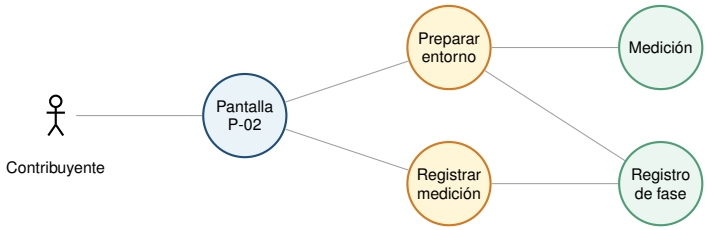


Figura 27: Diagrama de robustez de CU-001-005 — Escenario de evaluación.

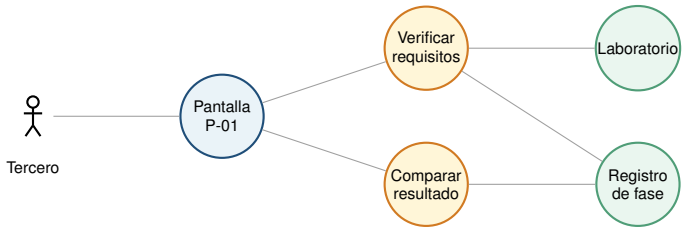


Figura 28: Diagrama de robustez de CU-001-006 — Reproducción por tercero.

Consistencia con el modelo de dominio. Las entidades que aparecen (*Entorno de trabajo*, *Issue*, *Evidencia*, *Pull Request*, *Medición*, *Laboratorio* y *Registro de fase*) son exactamente las clases del modelo de dominio de la Sección 10.5.7: el diagrama de robustez no introdujo ninguna entidad nueva, y las que el análisis sugirió se incorporaron al dominio antes de continuar. **Cobertura de cursos:** cada diagrama expone al menos un control para el curso básico y uno para el tratamiento de alternativos y excepciones, según las fichas de la Sección 10.8. **Una frontera por interfaz:** cada caso se conecta con una única pantalla del prototipo (Tabla 25); no hay casos con dos fronteras compitiendo por la misma interacción.

10.11 Diagramas de secuencia por caso de uso

Se presenta un diagrama por caso. Los participantes se derivan directamente de los controles y entidades del diagrama de robustez correspondiente, y cada mensaje corresponde a una oración del curso básico, de modo que el texto del caso puede recorrerse sobre el diagrama.

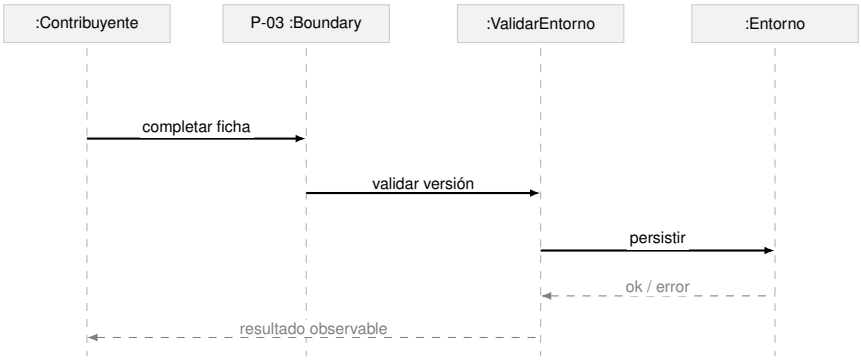


Figura 29: Diagrama de secuencia de CU-001-001 — Onboarding.

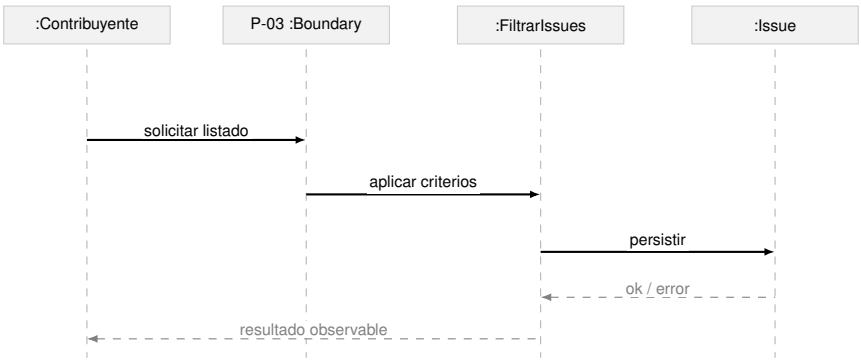


Figura 30: Diagrama de secuencia de CU-001-002 — Selección de issue.

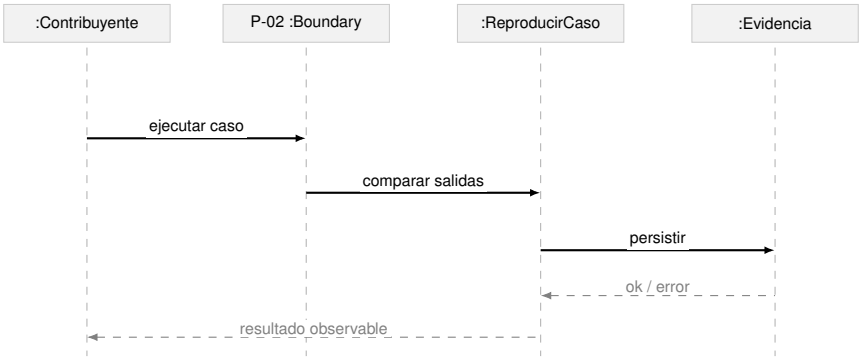


Figura 31: Diagrama de secuencia de CU-001-003 — Reproducción.

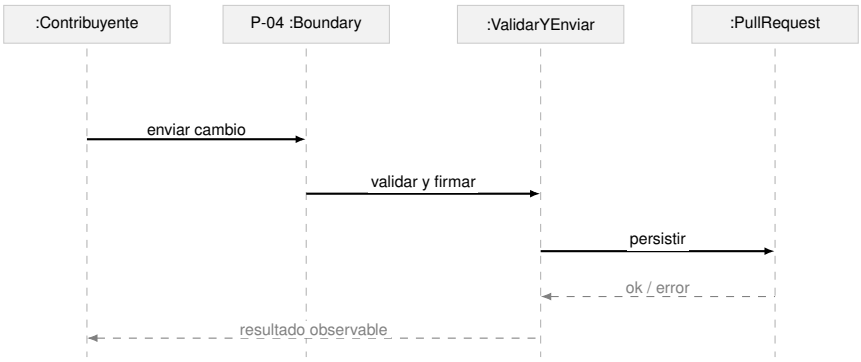


Figura 32: Diagrama de secuencia de CU-001-004 — Implementación y PR.

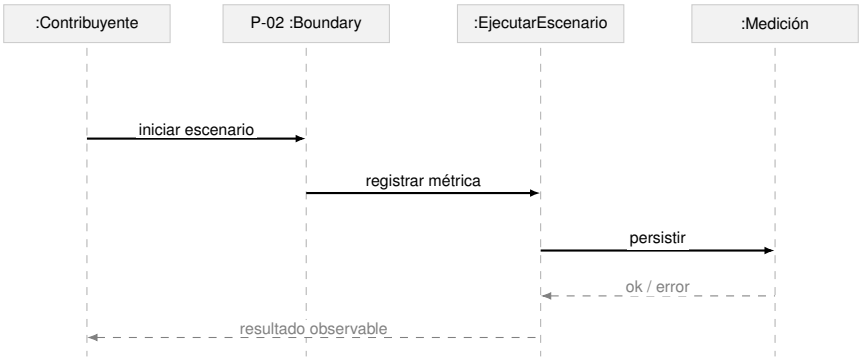


Figura 33: Diagrama de secuencia de CU-001-005 — Escenario de evaluación.

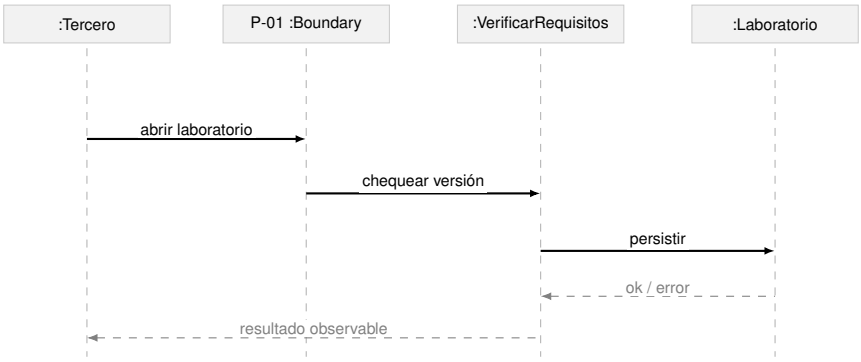


Figura 34: Diagrama de secuencia de CU-001-006 — Reproducción por tercero.

Errores y resultados observables. En los seis diagramas el retorno `ok / error` es explícito y el último mensaje devuelve al actor un resultado observable, en correspondencia con las excepciones E1 y E2 de las fichas. **Coherencia con arquitectura:** ningún mensaje salta de la frontera a la entidad sin pasar por un control, lo que mantiene la separación de capas declarada en la Sección 10.5.6 y evita el acoplamiento que los patrones de la Sección 10.13 buscan justamente controlar.

10.12 Diagrama de clases de diseño

El diagrama de diseño refina el modelo de dominio incorporando visibilidad, tipos, firmas de operaciones y multiplicidades.

Clase	Atributos (visibilidad: tipo)	Operaciones (firma)

`Contribucion`	`-id: String`
	`-issueRef: String`
	`-estado: EstadoContrib`
		`+registrar(i: Issue): void`
		`+actualizarEstado(e: EstadoContrib): void`
		`+esTrazable(): boolean`

`Issue`	`-numero: int`
	`-etiquetas: List<String>`
	`-asignado: boolean`
		`+esViable(c: Criterio): boolean`
		`+prioridad(): int`

`Laboratorio`	`-slug: String`
	`-versionBase: Version`
	`-verificadoEl: Date`
		`+requisitosCumplidos(e: Entorno): boolean`
		`+ejecutar(e: Entorno): Resultado`

`Entorno`	`-so: String`
	`-versionPodman: Version`
	`-rootless: boolean`
		`+compatibleCon(l: Laboratorio): boolean`

`Evidencia`	`-tipo: TipoEvidencia`
	`-salidaObtenida: String`
	`-salidaEsperada: String`
		`+coincide(): boolean`
		`+adjuntarA(i: Issue): void`

`RegistroFase`	`-fase: int` `-artefacto: String`	`+tieneArtefacto(): boolean`

Tabla 26: Clases de diseño con visibilidad, tipos y firmas.

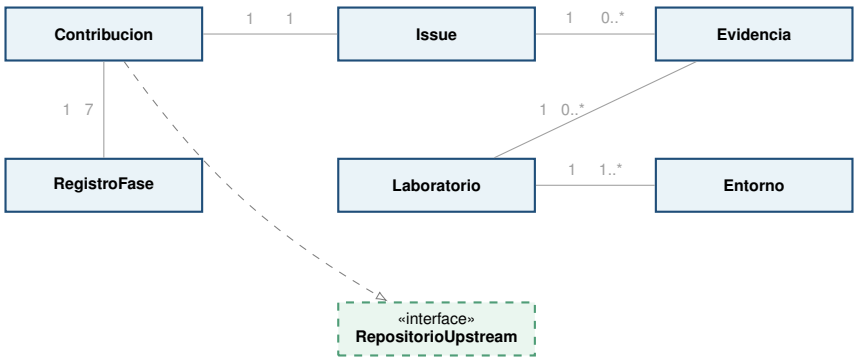


Figura 35: Clases de diseño, relaciones y multiplicidades. `Contribucion` depende de la interfaz `RepositorioUpstream`, no de su implementación.

Cohesión y acoplamiento. Cada clase tiene una única razón de cambio: `Laboratorio` decide sobre ejecución y compatibilidad, `Evidencia` sólo sobre comparación de salidas, `RegistroFase` sólo sobre completitud del marco. El acoplamiento se limitó extrayendo `RepositorioUpstream` como interfaz: la multiplicidad `1-7` entre `Contribucion` y `RegistroFase` expresa directamente RF-02, y la `1..*` entre `Laboratorio` y `Entorno` refleja que un laboratorio declara al menos un entorno soportado (RF-03). **Aislamiento de decisiones volátiles:** la interfaz encapsula lo único que puede cambiar por razones ajenas al proyecto, que es la API del repositorio upstream.

10.13 Patrones de diseño aplicados

Patrón	Problema concreto	Participantes y colaboración
Strategy	El criterio de priorización de issues cambia según el objetivo del ciclo (primera contribución, documentación, pruebas).	`FiltrarIssues` (contexto) delega en `Criterio` (estrategia); las variantes concretas implementan `esViable()`.
Adapter	El acceso al repositorio upstream depende de una API externa que el proyecto no controla.	`Contribucion` usa `RepositorioUpstream` (interfaz); el adaptador concreto traduce las llamadas a la API vigente.
Template Method	Las siete fases del marco tienen un esqueleto fijo pero pasos que varían por tipo de contribución.	Un método plantilla fija el orden de fases e invoca pasos redefinibles; `RegistroFase` verifica la completitud.

Tabla 27: Patrones aplicados, con el problema que resuelve cada uno.

Reflejo en clases y secuencias. *Strategy* aparece en la Figura 35 como la dependencia de `Issue` hacia `Criterio` en la firma `esViable(c: Criterio)`, y en el diagrama de secuencia de CU-001-002 como el mensaje `aplicar criterios`. *Adapter* es la relación punteada hacia `RepositorioUpstream` y el mensaje `persistir` de CU-001-004. *Template Method* se observa en la multiplicidad `1-7` y en la verificación `tieneArtefacto()`.

Patrón	Alternativa descartada y beneficio obtenido	Costo asumido
Strategy	Se descartó una cadena de condicionales; se gana poder agregar criterios sin tocar el contexto.	Una clase por criterio: más archivos para un caso hoy simple.
Adapter	Se descartó llamar la API directamente; se gana poder sustituirla y probar sin red.	Indirección adicional y una interfaz que mantener.
Template Method	Se descartó duplicar la secuencia por tipo; se gana un orden de fases único y verificable.	Herencia como acoplamiento: un cambio en el esqueleto impacta a todas las variantes.

Tabla 28: Alternativas, beneficios y costos de cada patrón.

10.14 Testing estructural

10.14.1 Unidad bajo prueba y dominio de entrada

La unidad seleccionada es `Laboratorio.requisitosCumplidos(e: Entorno)`, que decide si un entorno puede ejecutar un laboratorio. Se elige por ser la operación que concentra las decisiones de RF-03 y RNF-03 y la que un tercero ejercita al reproducir material (CU-001-006).

Elemento	Definición
Especificación	Devuelve verdadero si y sólo si el entorno declara sistema Linux soportado, versión de Podman mayor o igual a la base, y modo *rootless* cuando el laboratorio lo exige.
Dominio de entrada	`e.so` ∈ {soportado, no soportado}; `e.versionPodman` ∈ {menor, igual, mayor}; `e.rootless` ∈ {verdadero, falso}; `l.exigeRootless` ∈ {verdadero, falso}.
Salida	`boolean`, más el motivo del rechazo cuando corresponde.

Tabla 29: Unidad bajo prueba, especificación y dominio de entrada.

10.14.2 Grafo de flujo de control

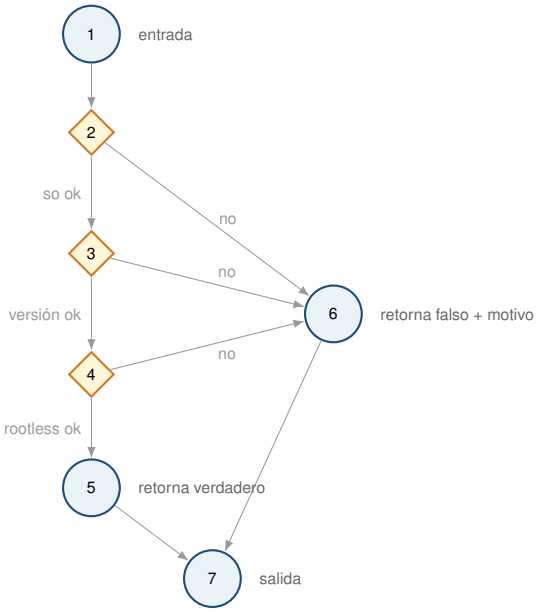


Figura 36: Grafo de flujo de control de `requisitosCumplidos`. Nodos 2, 3 y 4 son decisiones; aristas: 9; nodos: 7.

La complejidad ciclomática resulta $V(G) = A - N + 2 = 9 - 7 + 2 = 4$, lo que fija en cuatro la cantidad mínima de caminos independientes a cubrir.

10.14.3 Criterio de cobertura y conjunto de pruebas

Criterio declarado: cobertura de decisión (rama), con el agregado de recorrer los cuatro caminos independientes derivados de $V(G)$. Se elige cobertura de rama y no sólo de sentencia porque los tres nodos de decisión concentran toda la lógica de la unidad y la cobertura de sentencia podría satisfacerse sin ejercitar las ramas negativas.

Prueba	Entrada	Salida esperada	Camino / clase
TP-01	so soportado, versión mayor, rootless exigido y presente	verdadero	1-2-3-4-5-7 (normal)
TP-02	so no soportado	falso + motivo so	1-2-6-7 (falla)
TP-03	so soportado, versión menor	falso + motivo versión	1-2-3-6-7 (falla)
TP-04	so soportado, versión igual, rootless exigido y ausente	falso + motivo rootless	1-2-3-4-6-7 (límite)
TP-05	so soportado, versión igual a la base, rootless no exigido	verdadero	1-2-3-4-5-7 (límite)

Tabla 30: Conjunto de pruebas. TP-01 a TP-04 cubren los cuatro caminos independientes; TP-05 ejercita el valor límite de versión.

10.14.4 Cobertura medida y trazabilidad

Métrica	Resultado	Evidencia
Cobertura de sentencia	7/7 nodos	Informe de cobertura del ejecutor de pruebas.
Cobertura de decisión	6/6 ramas	Idem; ambas salidas de los nodos 2, 3 y 4.
Caminos independientes	4/4	TP-01 a TP-04, Tabla 30.
Re-ejecución	automática	Las pruebas corren en el pipeline junto con CP-003 y CP-004 de la Sección 10.5.10.

Tabla 31: Cobertura medida sobre la unidad bajo prueba.

Trazabilidad completa. La cadena queda cerrada en los dos sentidos: RF-03 y RNF-03 → CU-001-006 (Tabla 24) → control *Verificar requisitos* (Sección 10.10) → operación `requisitosCumplidos` (Tabla 26) → pruebas TP-01 a TP-05 (Tabla 30). Desde cualquier prueba puede recorrerse el camino inverso hasta el requerimiento que la origina.

10.15 Registro de hitos RRR, PDR y CDR

ICONIX organiza el avance en tres revisiones formales. Los tres hitos fueron revisados y aprobados por la dirección del trabajo. El detalle de qué artefactos somete cada uno se conserva para trazabilidad de la revisión.

Hito	Artefactos que somete a revisión	Estado	Responsable de aprobar
RRR	Requerimientos, criterios de aceptación, matriz de trazabilidad, casos de uso y prototipo.	**Revisado y aprobado** (22-09-2026).	Dirección de Investigación.
PDR	Diagramas de robustez y modelo de dominio actualizado.	**Revisado y aprobado** (22-09-2026).	Dirección de Investigación.
CDR	Diagramas de secuencia, clases de diseño, patrones y testing estructural.	**Revisado y aprobado** (22-09-2026).	Dirección y cátedra.

Tabla 32: Estado de los hitos de revisión.

La aprobación de los tres hitos queda asentada en la ficha de seguimiento de la página 3. Los artefactos sometidos a cada revisión son los que se detallan en la Tabla 32.

C. Anexos (Optativo)

11. Anexos

11.1 Normas de Calidad

- **Documentación:** cada material didáctico publicado incluye versión de Podman, fecha de revisión y licencia.
- **Código:** ejemplos en Go siguen `gofumpt` y validación con `golangci-lint`, consistente con `.golangci.yml` de Podman.
- **Contribuciones upstream:** respetan `CONTRIBUTING.md`, `LLM_POLICY.md` y DCO (`git commit -s`).
- **Revisión académica:** papers revisados por la Dirección de Investigación antes de submission.
- **Trazabilidad:** cada entrega genera commit firmado en el repositorio Git del proyecto.

11.2 Anexos específicos del proyecto

Esta Segunda Entrega se distribuye como un paquete ZIP con el documento principal, el informe de seguimiento y los antecedentes de la línea. Los nombres listados corresponden a los archivos contenidos en `Torassa_Valentin_E2-2C_v1.zip`:

1. `Torassa_Valentin_E2-2C_v1.pdf` – Segunda Entrega del segundo cuatrimestre (documento principal).
2. `Torassa_Valentin_CANVAS_v1.pdf` – Business Model Canvas (entrega previa, adjunto como referencia).
3. `Anexo_Vinculacion_Tecnologica.pdf` – Anexo conceptual de la línea.
4. `WICC_2026_Paper_Go.pdf` – Paper presentado en WICC sobre Go como lenguaje de diseño.
5. `WICC_2026_Certificado.pdf` – Certificado de aprobación WICC 2026.
6. `WICC_2026_Poster_Go.pdf` – Póster de la ponencia WICC.
7. `CACIC_2026_Contribuciones_Software_Libre_Podman.pdf` – Borrador para CACIC sobre contribuciones a SL (caso Podman).

8. `CACIC_2026_Analisis_Funcional_Podman.pdf` – Borrador para CACIC sobre análisis funcional de Podman.
9. `CACIC_2026_Parches_Seguridad_Ubuntu.pdf` – Borrador para CACIC sobre parches de seguridad comunitarios en Ubuntu (línea `vt-security-fixes`).
10. `seguimiento-checklist.md` – Ubicación de las evidencias para la checklist tecnológica y el punto 7 de esta entrega.
11. `README.md` – Guía breve de contenido y estado de la entrega.

Los documentos 3 a 9 son optativos y se incluyen como respaldo de la línea de investigación que sostiene el plan.

11.3 Evidencia upstream: contribuciones al repositorio Podman

Este anexo registra las contribuciones efectivas y planificadas al repositorio `containers/podman`. Complementa el Plan de Desarrollo Tecnológico con evidencia verificable externamente y una lista de posibles aportes futuros.

Contribución mergeada – PR #28637

Ítem	Detalle
Título	Add `-retry` support to manifest push.
Número	#28637.
URL	`https://github.com/containers/podman/pull/28637`
Autor	Valentín Torassa Colombero (`ValentinTorassa`).
Fecha de apertura	3 de mayo de 2026.
Estado	**Mergeado** el 23 de junio de 2026 por el mantenedor Paul Holzinger (`Luap99`); *merge commit* `4ae1898`.
Etiqueta	`kind/api-change`.
Métricas de cambio	87 líneas agregadas, 10 eliminadas, 9 archivos modificados (1 *commit*).
Issue que resuelve	#28590 “Podman-manifest-push have no retry option” [9] – `Good First Issue`, `triaged`.
Alcance funcional	Habilita `-retry` y `-retry-delay` en `podman manifest push`, en paridad con `podman push`. Cubre CLI local, cliente remoto, REST API (query params `retry` y `retryDelay`), manpages, Swagger y pruebas e2e.
Fases del marco aplicadas	1 a 6 completadas (Onboarding a PR + revisión); fase 7 (Reflexión) documentada tras el *merge*.

Por qué es relevante para el plan. El PR demuestra que el marco metodológico ya genera contribuciones de calidad upstream, no sólo documentación. Cubre múltiples capas del repositorio (CLI, dominio, API REST, bindings, docs y tests), lo que evidencia comprensión transversal de la arquitectura descrita en [6].

Contribuciones planificadas (pipeline)

Las siguientes líneas de contribución se preparan para los próximos avances, en consistencia con el roadmap del proyecto y con las áreas de menor riesgo identificadas en el primer borrador CACIC:

1. **Documentación rootless:** mejoras puntuales de manpages y ejemplos, en particular sobre limitaciones (subuids/subgids, NFS, cgroups v1) y diagnóstico de fallos típicos.
2. **Pruebas de regresión para pods:** casos BATS sobre publicación de puertos a nivel de pod versus contenedor, comunicación por `localhost` y resolución DNS interna.

3. **Quadlet:** ejemplos y validaciones de archivos declarativos para servicios persistentes académicos y de pyme.
4. **Mensajes de error accionables:** transformar fallas técnicas en sugerencias útiles para usuarios nuevos, sobre todo en escenarios rootless.
5. **Compatibilidad CLI Podman/Docker:** revisión de `podman manifest` y comandos relacionados para cerrar gaps de paridad con flujos Docker.

Cada contribución producirá una entrada en este anexo en los avances posteriores, con el mismo formato tabular del PR #28637.

Cumplimiento de políticas de Podman

- **DCO:** todos los commits firmados con `git commit -s`.
- **Comunicación en palabras propias** [4]: descripción del PR, mensajes de commit y respuestas a revisores redactados por el contribuyente; ningún texto LLM publicado verbatim en issues, PRs ni comentarios.
- **Calidad técnica:** cumplimiento de `gofumpt`/`golangci-lint`, `make validatepr` en verde, pruebas asociadas al cambio.
- **No edición de `vendor/`:** cualquier dependencia se gestiona con `go get` y `make vendor`.

11.4 Bibliografía Consultada

[1] Open Container Initiative. Image format specification. `https://github.com/opencontainers/image-spec`, 2023.

[2] Open Container Initiative. Runtime specification. `https://github.com/opencontainers/runtime-spec`, 2023.

[3] Podman Project. containers/podman – a tool for managing oci containers and pods. `https://github.com/containers/podman`, 2026. Repositorio analizado en rama main, commit 9f02e222, 2026-05-01.

[4] Podman Project. Podman llm (ai) development policy. `https://github.com/containers/podman/blob/main/LLM_POLICY.md`, 2026.

[5] Valentín Torassa Colombero. Add `-retry` support to manifest push (pr #28637). Pull Request a `containers/podman`, `https://github.com/containers/podman/pull/28637`, 2026. Abierto el 2026-05-03; 86 líneas agregadas, 10 eliminadas, 9 archivos. Resuelve issue #28590. Estado al cierre de E1: en revisión.

[6] Valentín Torassa Colombero, María Eugenia Casco, and Santiago Enrique Roatta. Análisis funcional de podman para entornos de contenedores rootless: Arquitectura, operación y líneas de contribución. Borrador presentado al Workshop de Arquitectura, Redes y Sistemas Operativos (WARSO) del CACIC 2026, 2026.

[7] Valentín Torassa Colombero, María Eugenia Casco, and Santiago Enrique Roatta. Contribuciones a proyectos de software libre como estrategia de formación en ingeniería de software: un marco metodológico aplicado al ecosistema podman. Borrador presentado al Workshop de Ingeniería de Software (WIS) del CACIC 2026, 2026.

[8] Valentín Torassa Colombero, Santiago Enrique Roatta, and María Eugenia Casco. Go como lenguaje de diseño: Reducción de complejidad accidental en sistemas concurrentes. In *Workshop de Investigadores en Ciencias de la Computación (WICC)*, Argentina, 2026.

[9] Jean-Baptiste Trystram. Podman-manifest-push have no retry option (issue #28590). Issue en `containers/podman`, `https://github.com/containers/podman/issues/28590`, 2026. Etiquetas: Good First Issue, kind/feature, triaged.